Gerência de Memória

Prof. Paulo Roberto O. Valim
Universidade do Vale do Itajaí

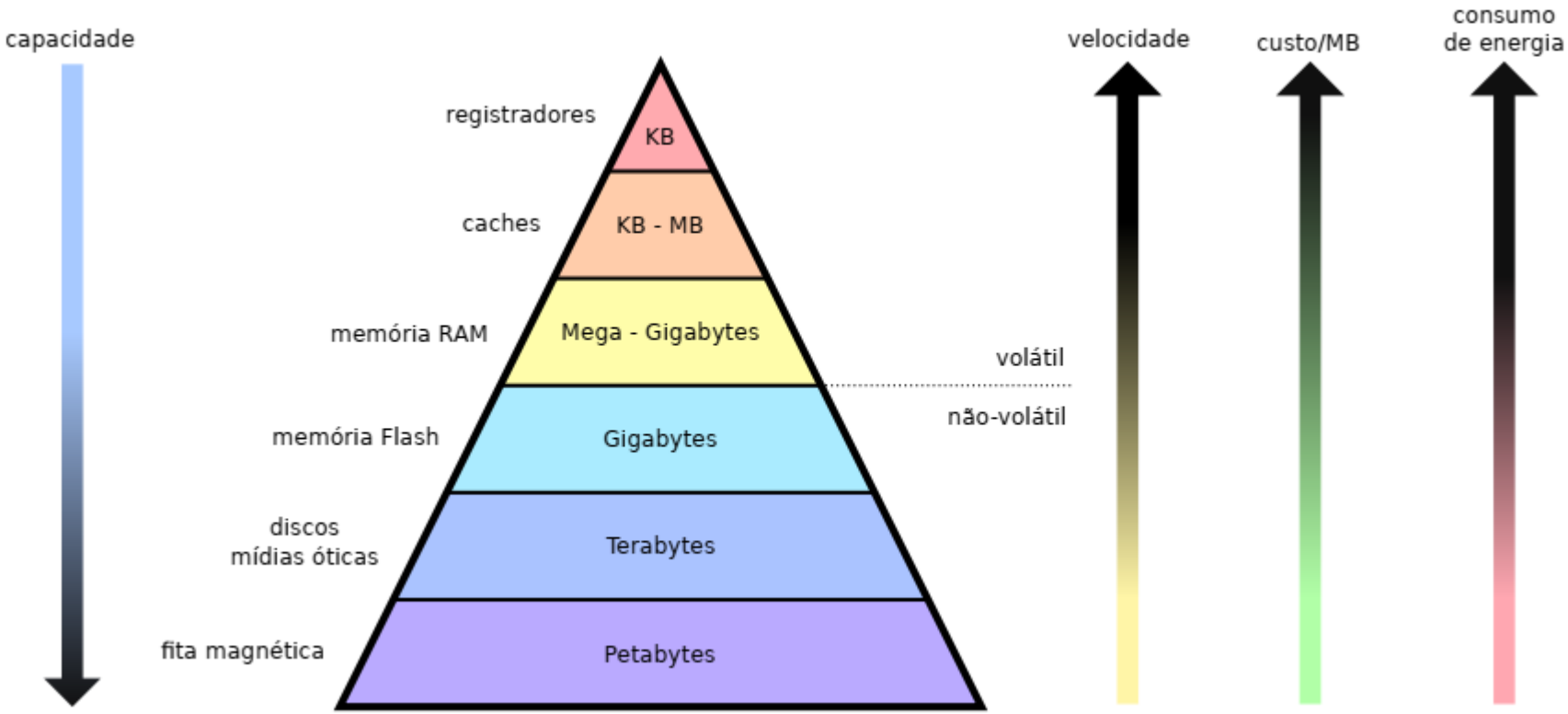
Hardware de Memória

- A memória principal é um componente fundamental em qualquer sistema de computação

- O hardware de memória pode ser bastante complexo, envolvendo diversas estruturas, como memórias RAM, caches, unidade de gerência, barramentos, etc, o que exige um esforço de gerência significativo por parte do sistema operacional.

Tipos de Memória

- Existem diversos tipos de memória em um sistema de computação, cada um com suas próprias características e particularidades, mas todos com um mesmo objetivo: armazenar informação.

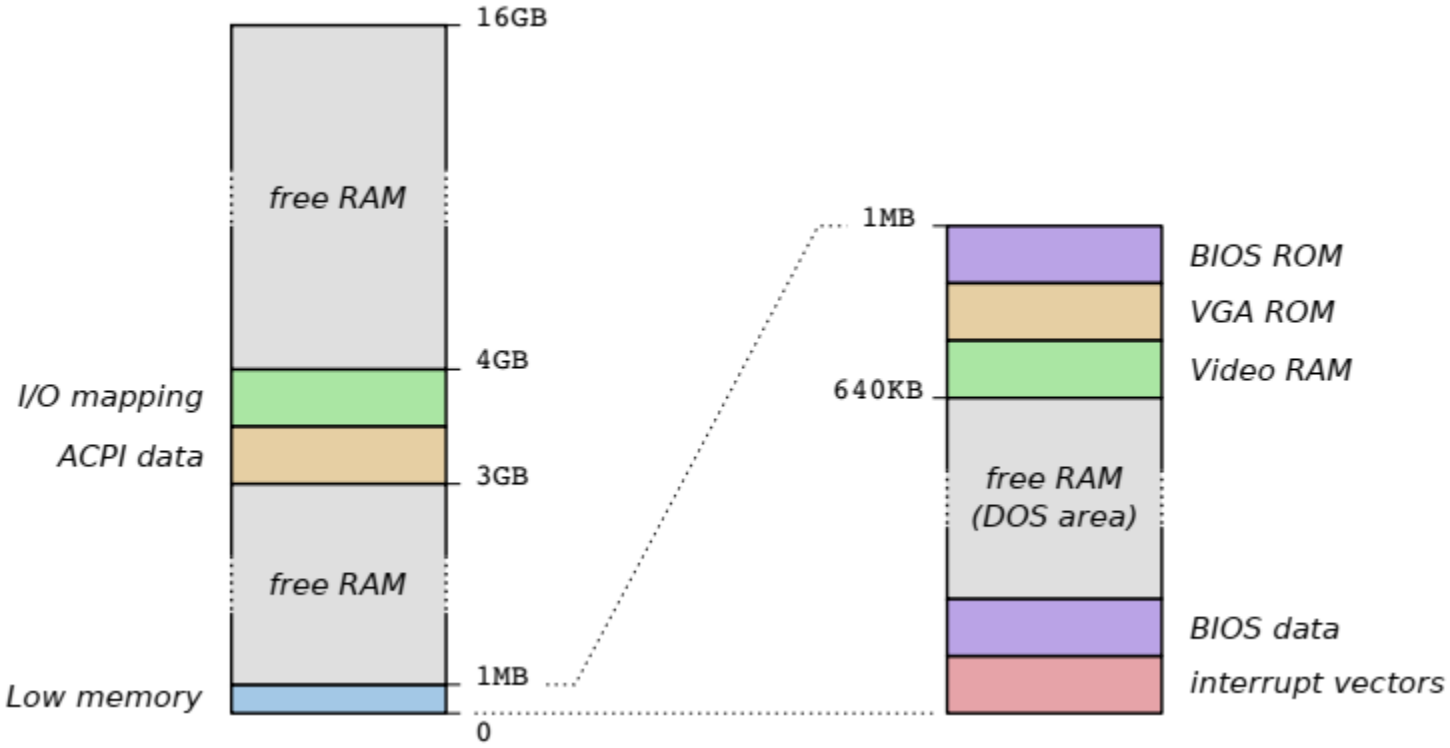


Hierarquia de Memória

Ex. de tempos de acesso e taxa de transferência

- **Cache L2**
  - **Tempo de acesso:** ~1 a 10 nanosegundos (ns)
  - **Taxa de transferência:** ~10 a 100 GB/s
- **Memória RAM**
  - **Tempo de acesso:** ~10 a 100 nanosegundos (ns)
  - **Taxa de transferência:** ~10 a 50 GB/s
- **Disco SSD**
  - **Tempo de acesso:** ~10 a 100 microssegundos (µs)
  - **Taxa de transferência:** ~500 MB/s a 3 GB/s (para SSDs NVMe, pode ser maior, até ~7 GB/s)
- **Disco SATA (HDD)**
  - **Tempo de acesso:** ~10 a 20 milissegundos (ms)
  - **Taxa de transferência:** ~100 MB/s a 200 MB/s
- **DVD ROM**
  - **Tempo de acesso:** ~100 a 200 milissegundos (ms)
  - **Taxa de transferência:** ~1.4 MB/s (para velocidade de leitura de 1x) a ~22 MB/s (para velocidade de leitura de 16x)

Layout da Memória RAM (física) de um computador



Os sistemas atuais mais sofisticados usam os conceitos de espaços de endereçamento e de memória virtual, vistos nas próximas seções, para desacoplar a visão da memória pelos processos da estrutura física da memória no hardware.

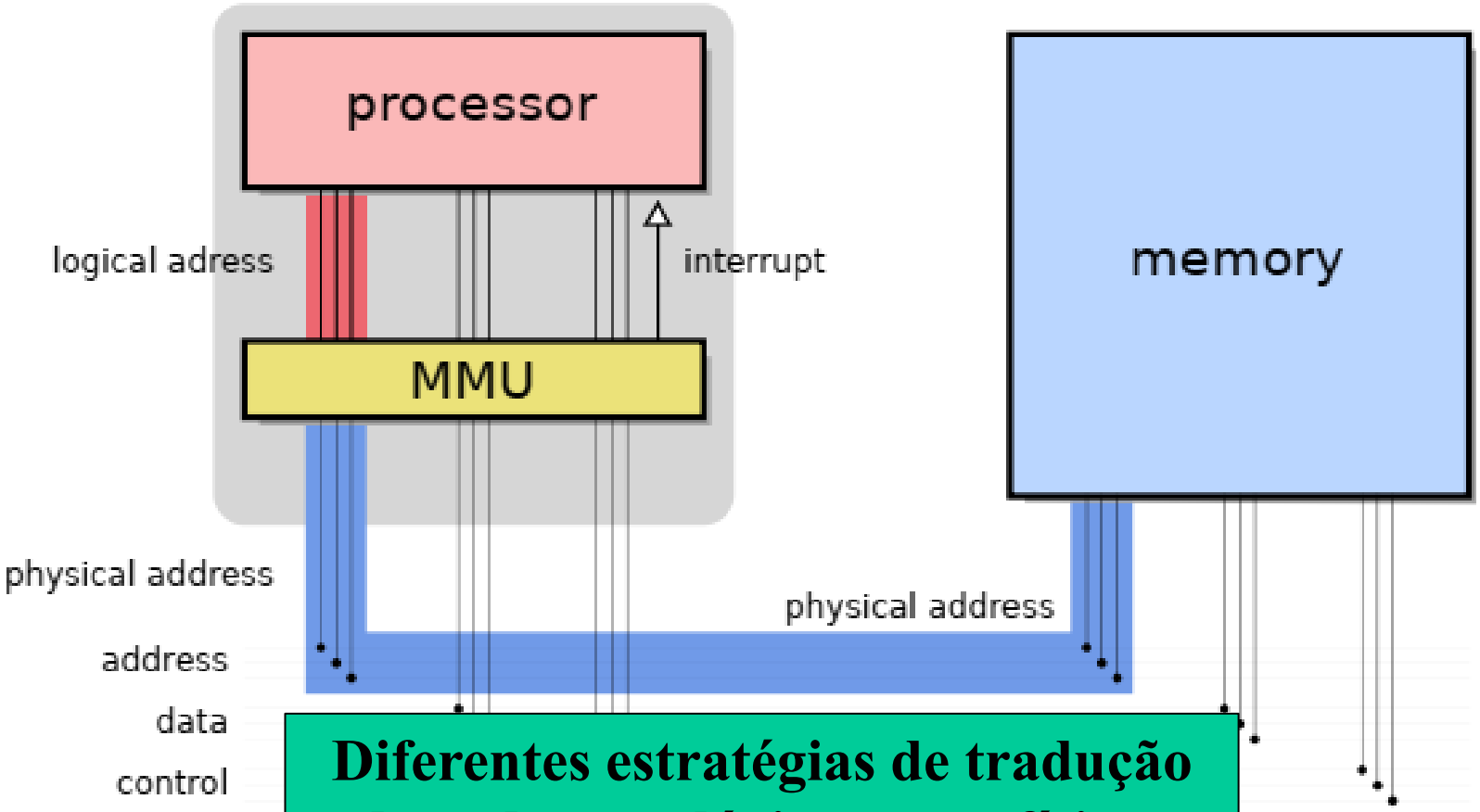
Espaço de endereçamento de Memória Virtual

- O processador acessa a memória RAM através de barramentos de dados, de endereços e de controle. O barramento de endereços (como os demais) possui um número fixo de vias, que define a quantidade total de endereços de memória que podem ser gerados pelo processador: um barramento de dados com *n* vias consegue gerar 2^n endereços distintos. O conjunto de endereços de memória que um processador pode produzir é chamado de **espaço de endereçamento**.

- Para ocultar a organização complexa da memória física e simplificar os procedimentos de alocação da memória aos processos, os sistemas de computação modernos implementam a noção de **memória virtual**, na qual existem dois tipos de endereços de memória **distintos**:
  - Endereços físicos
  - Endereços lógicos (ou virtuais)

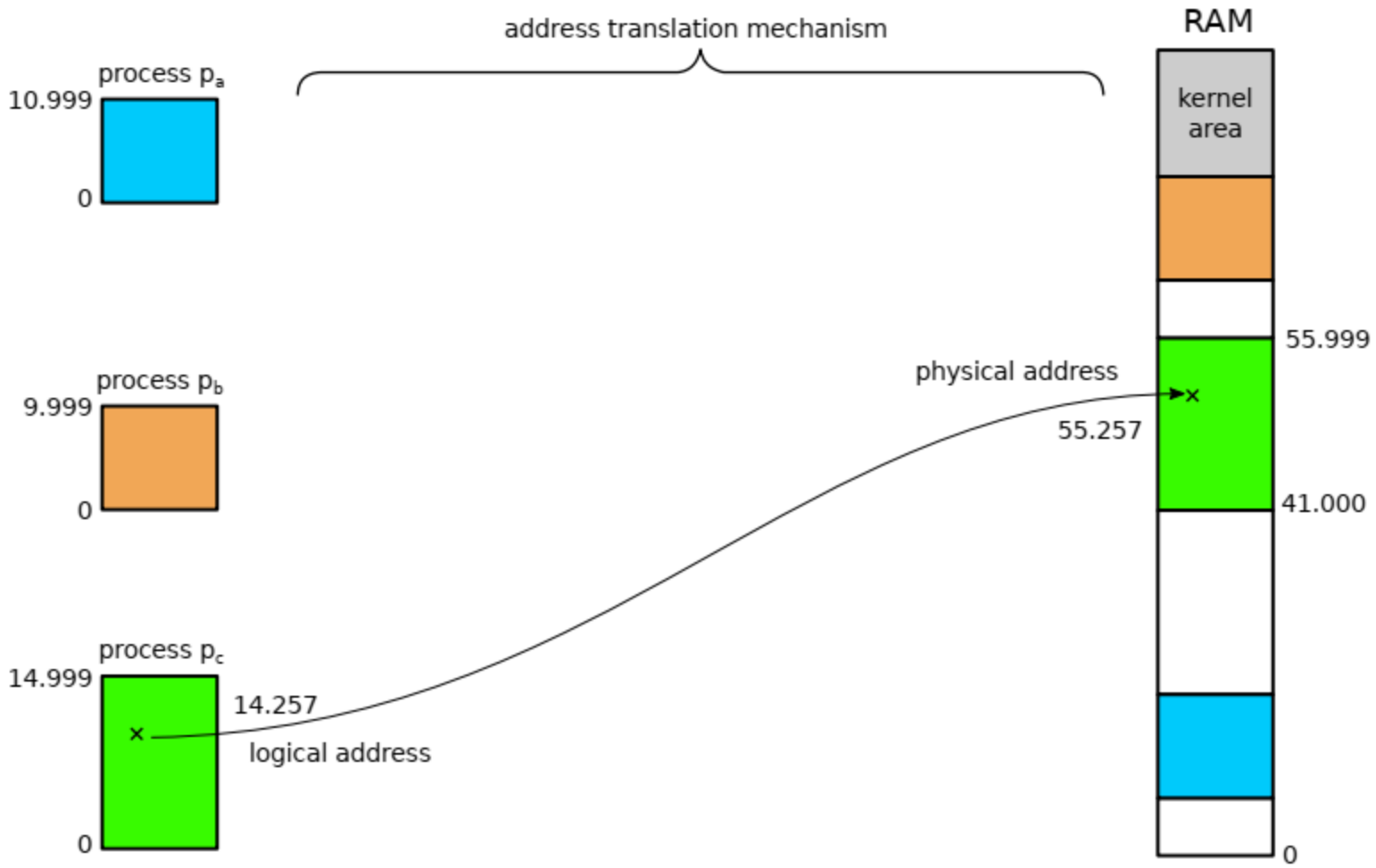
- Por questões de desempenho, a tradução de endereços lógicos em físicos é feita por um componente específico do hardware do computador, denominado **Unidade de Gerência de Memória** (MMU – Memory Management Unit).

Unidade de Gerência de Memória (MMU)

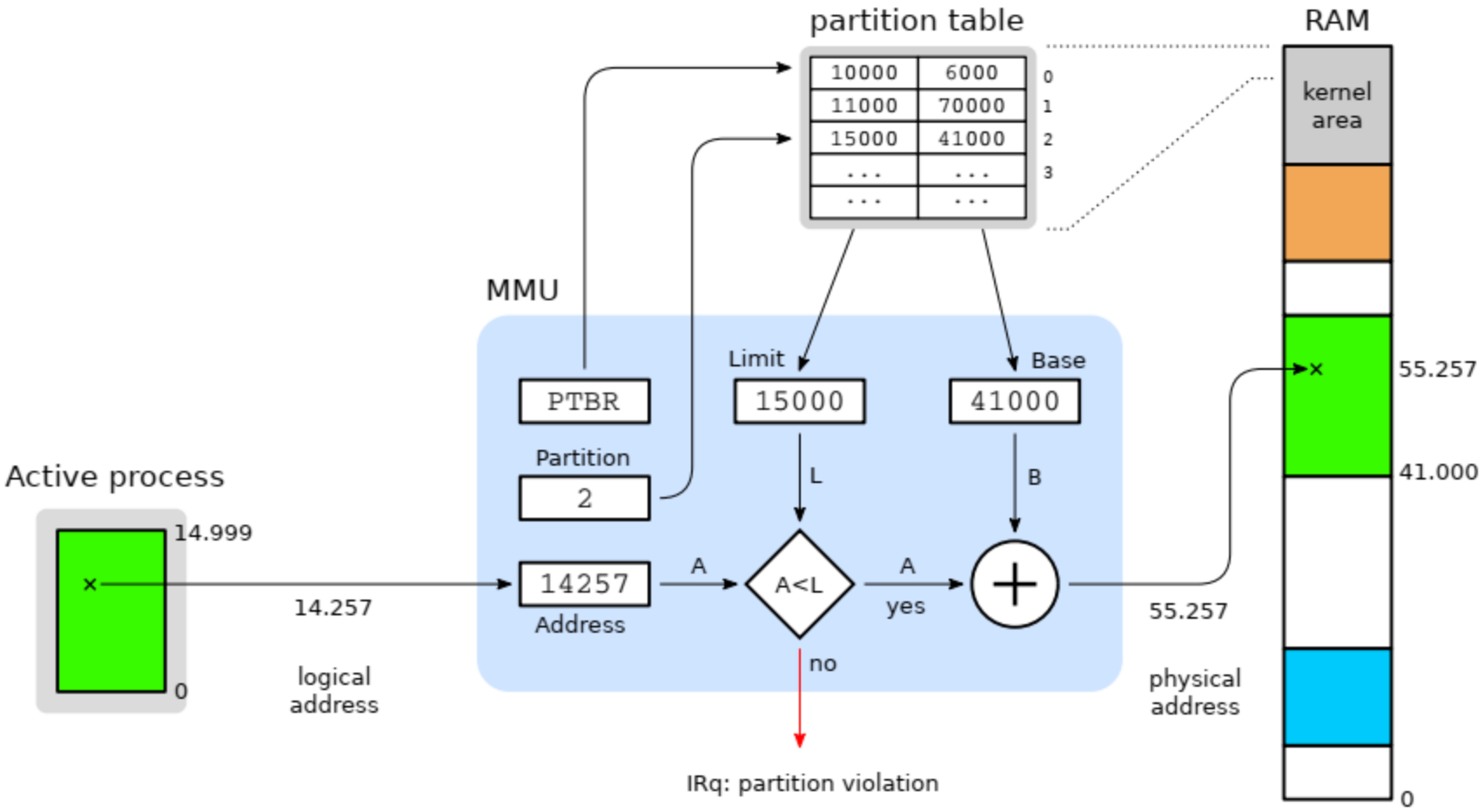


Diferentes estratégias de tradução de endereços lógicos para físicos usadas pelas MMU's

Memória Virtual por Partições

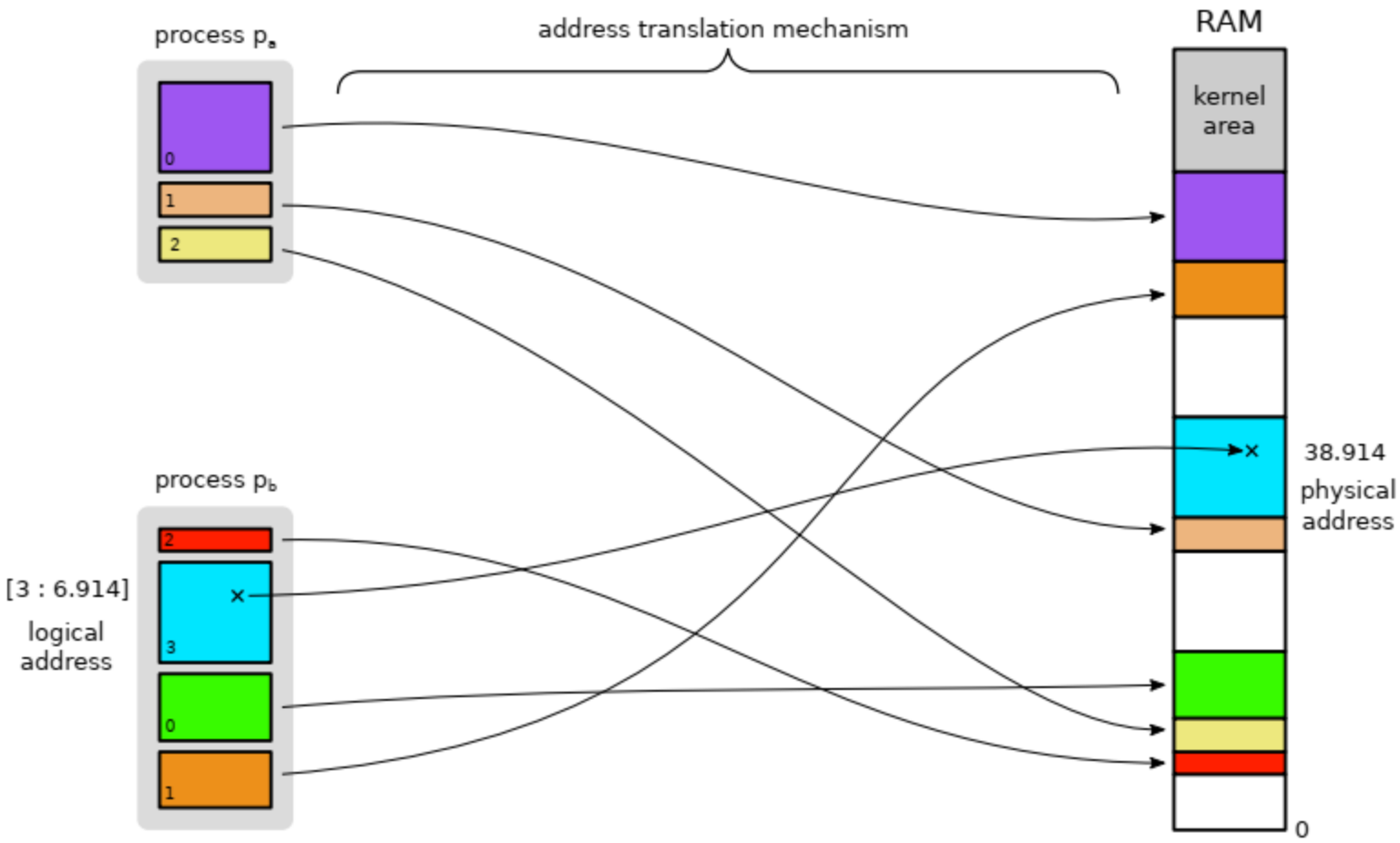


MMU com partições

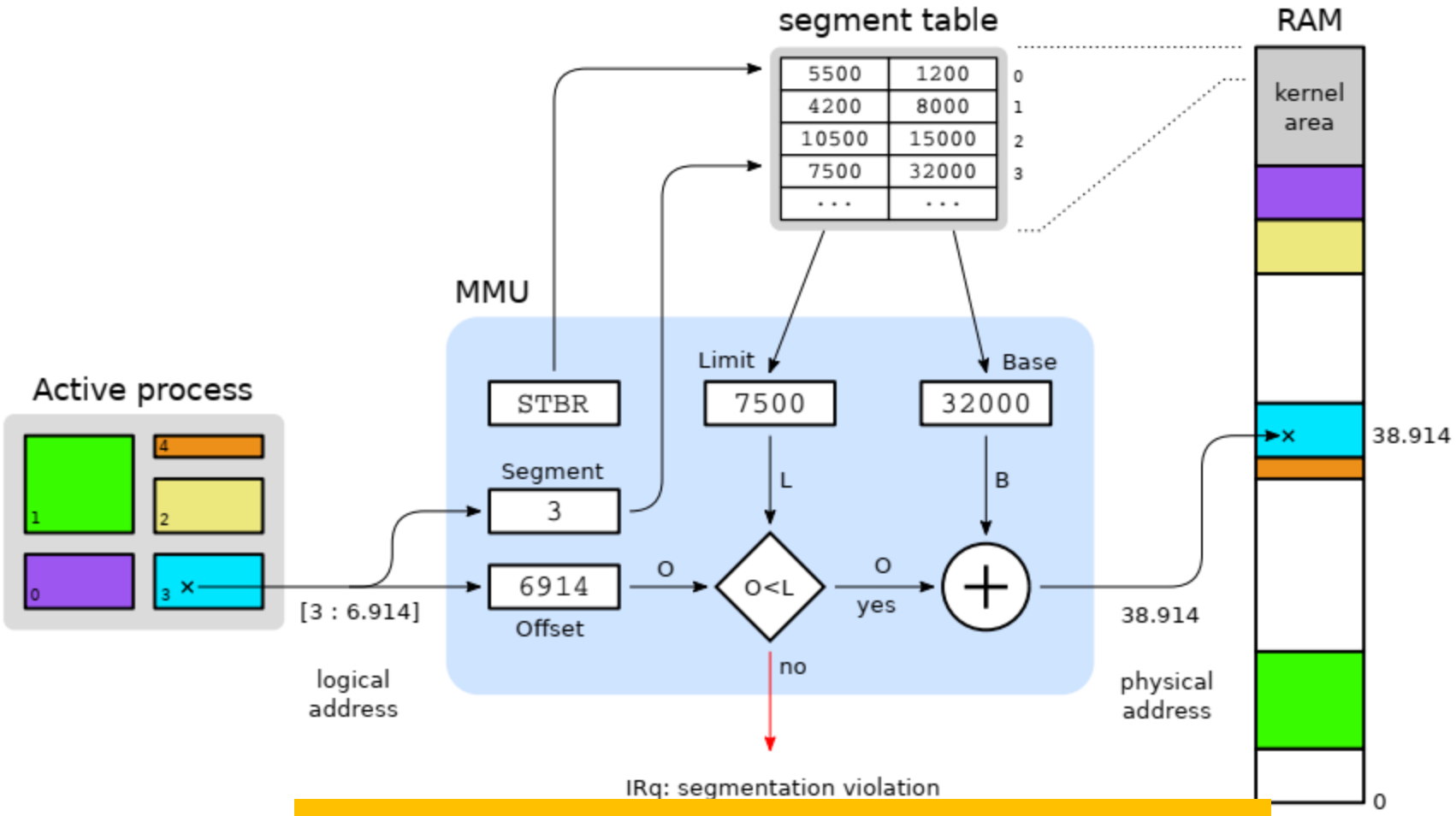


Está sujeita a fragmentação externa

Memória Virtual por segmentos

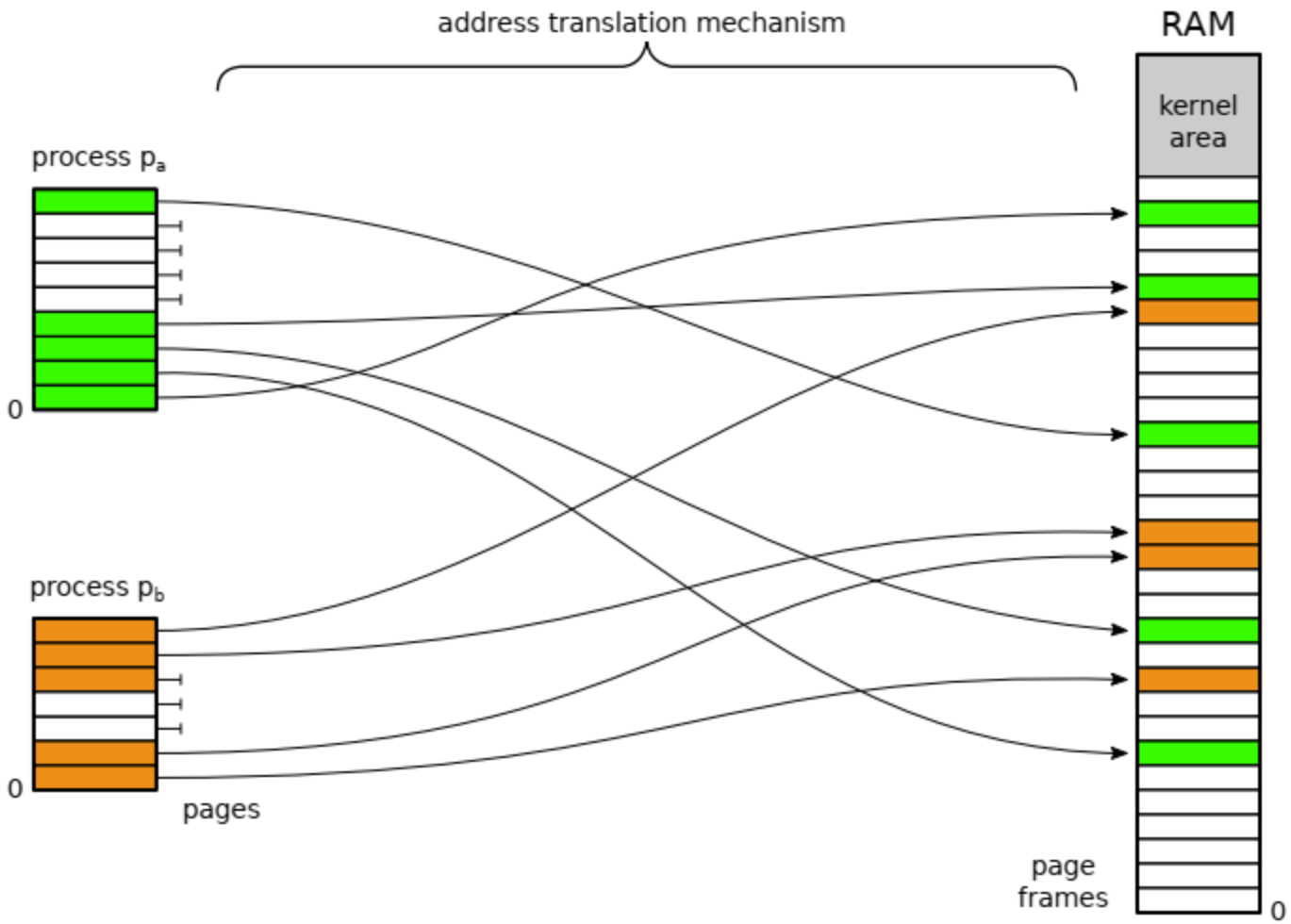


MMU com segmentação



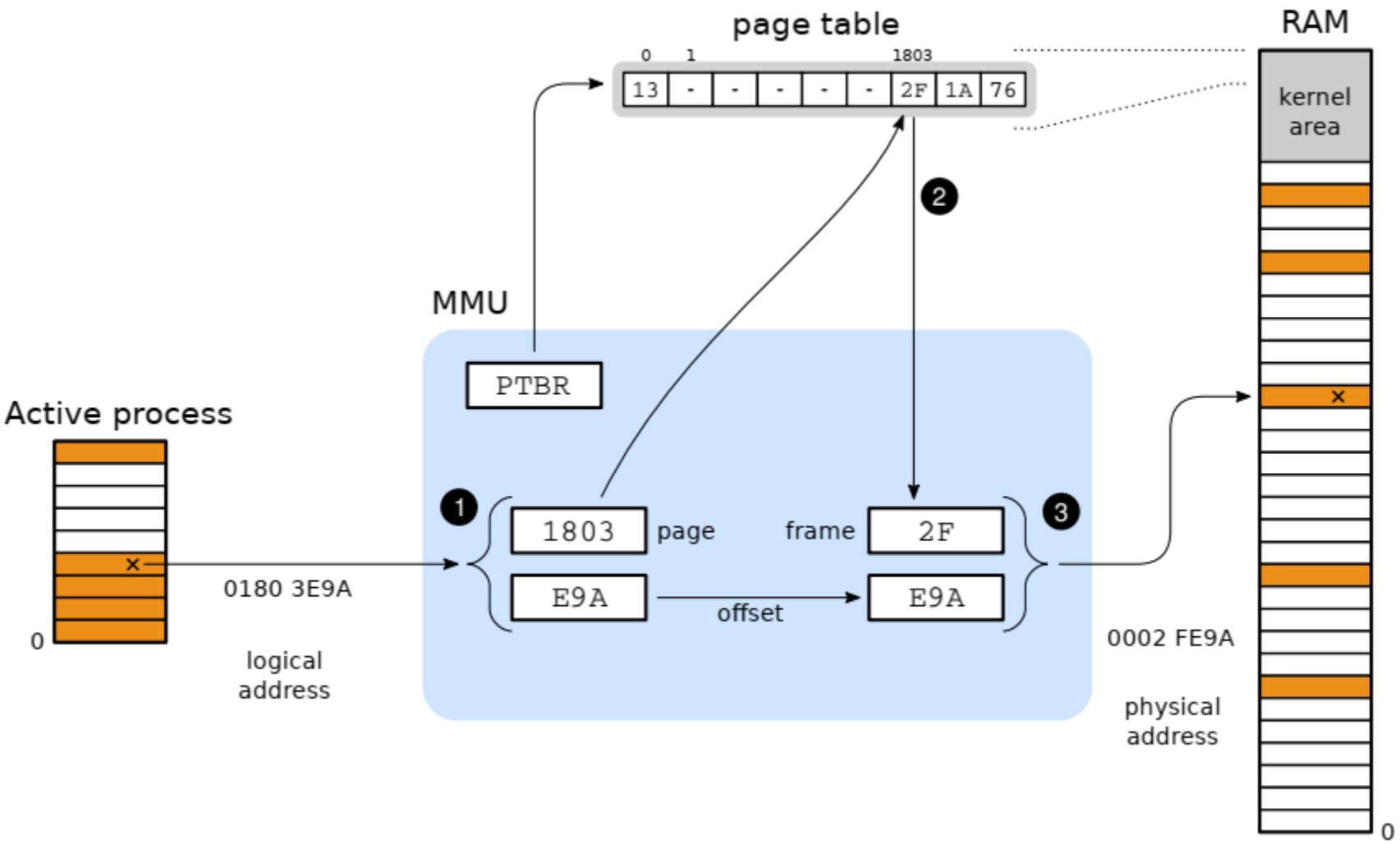
Está sujeita a fragmentação externa

Memória Virtual por Páginas



Está sujeita a fragmentação interna

MMU com paginação



MMU com paginação – Flags de Status e Controle

• *Valid*: indica se a página é válida, ou seja, se existe no espaço de endereçamento daquele processo; se este bit estiver em 0, tentativas de acesso à página irão gerar uma interrupção de falta de página (*page fault*);

• *Writable*: controla se a página pode ser acessada em leitura e escrita (1) ou somente em leitura (0);

• *User*: se estiver ativo (1), código executando em modo usuário pode acessar a página; caso contrário, a página só é acessível ao núcleo do sistema;

• *Present*: indica se a página está presente na memória RAM ou se foi transferida para um armazenamento secundário, como ocorre nos sistemas com paginação em disco (Seção 17.2);

• *Accessed*: indica se a página foi acessada recentemente; este bit é ativado pela MMU a cada acesso à página e pode ser desativado pelo núcleo quando desejado; essa informação é usada pelos algoritmos de paginação em disco;

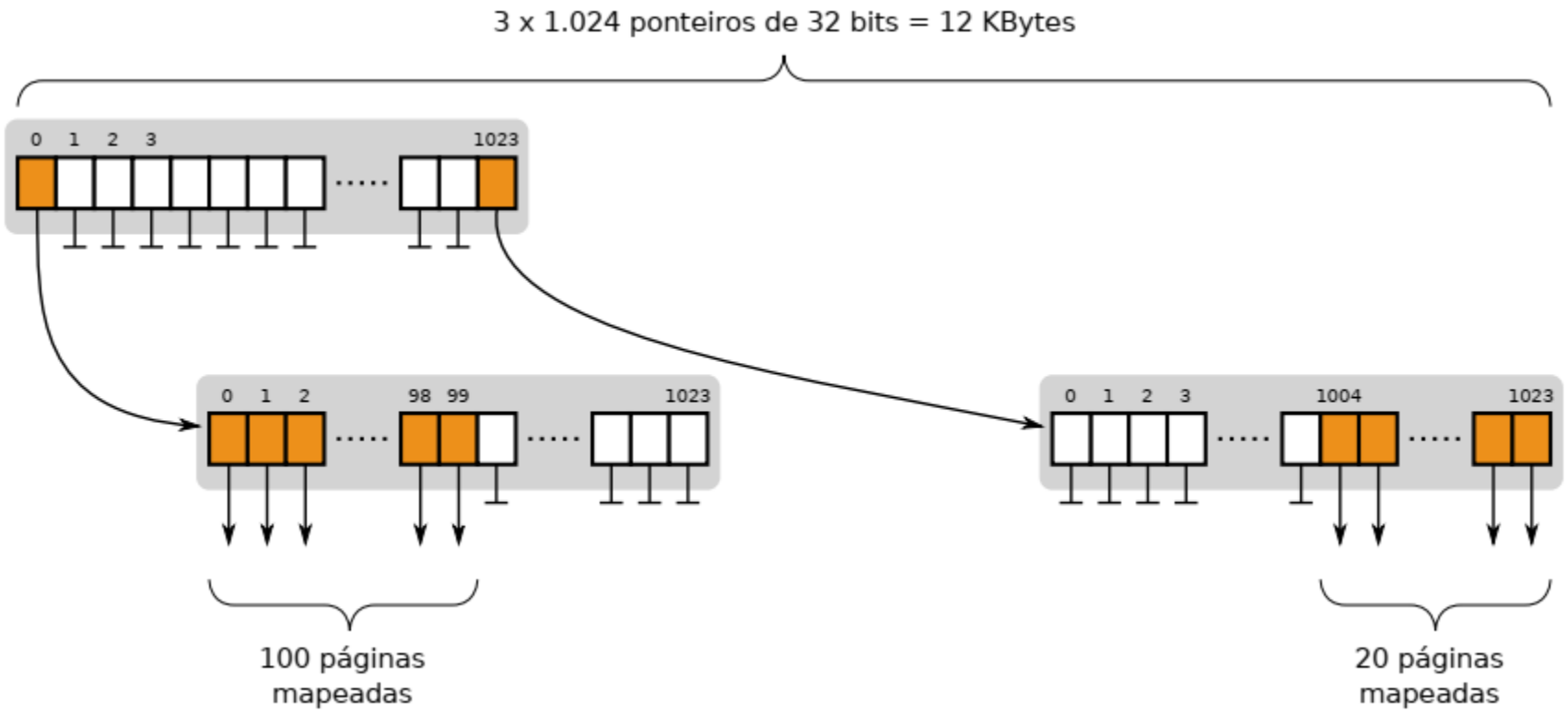
• *Dirty*: este bit é ativado pela MMU após uma escrita na página, para informar que ela foi modificada (que foi "suja"); também é usado pelos algoritmos de paginação em disco.

Paginação multinível

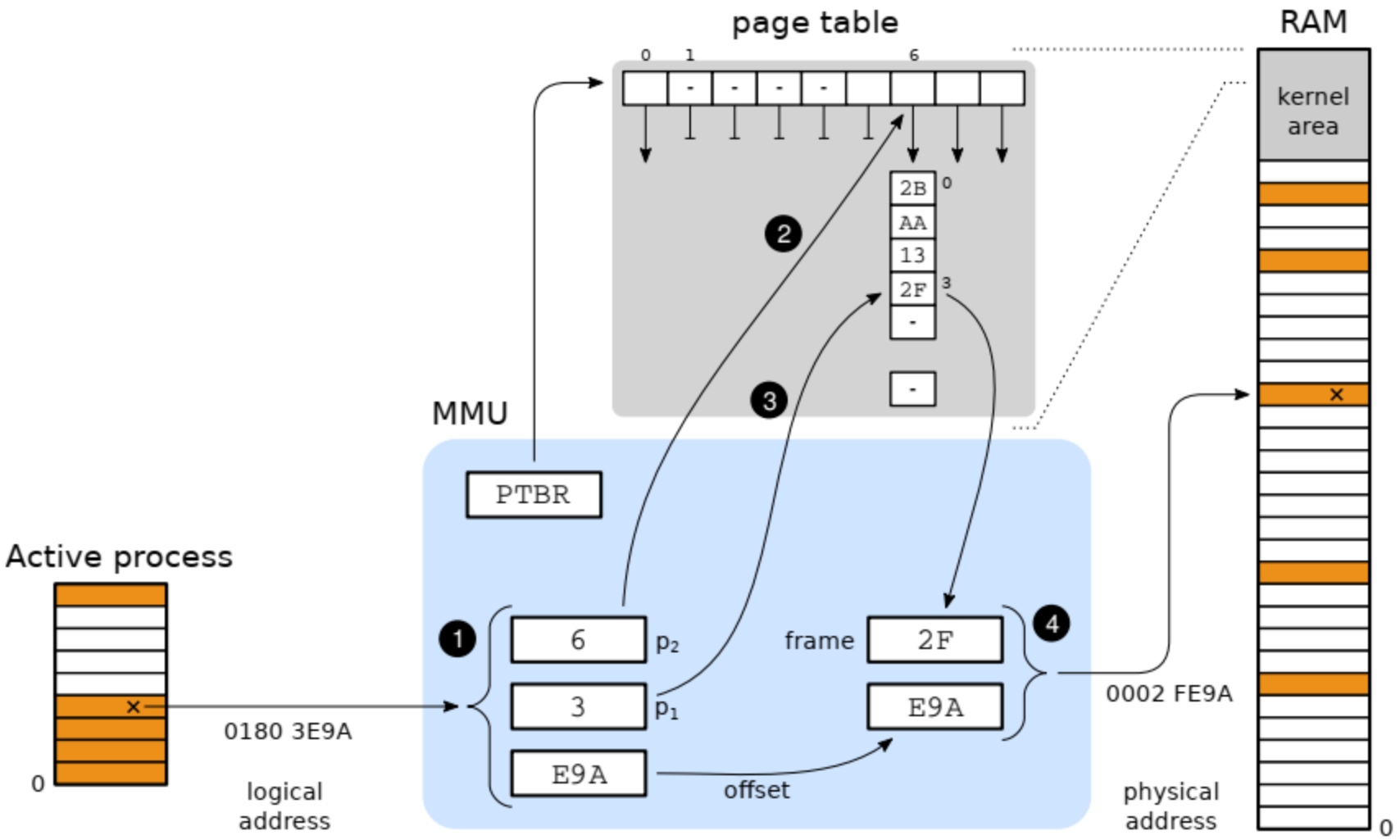
- o processador Intel 80386 usa endereços lógicos de 32 bits e páginas com 4 KBytes , cada entrada na tabela de páginas ocupa cerca de 32 bits, ou 4 bytes (20 bits para o número de quadro e os 12 bits restantes para informações e *flags* de controle). Considerando que cada tabela de páginas tem 2^{20} páginas, uma tabela ocupará 4 MBytes de memória (4×2^{20} bytes) se for armazenada de forma linear na memória. No caso de processos pequenos, com muitas páginas não mapeadas, uma tabela de páginas linear poderá ocupar mais espaço na memória que o próprio processo

- Para resolver esse problema, são usadas *tabelas de páginas multiníveis*, estruturadas na forma de árvores: uma primeira tabela de páginas (ou *diretório de páginas*) contém ponteiros para tabelas de páginas secundárias e assim por diante, até chegar à tabela que contém o número do quadro desejado. Quando uma tabela secundária não contiver entradas válidas, ela não precisa ser alocada; isso é representado por uma entrada nula na tabela principal.

Paginação multinível



MMU com paginação multinível

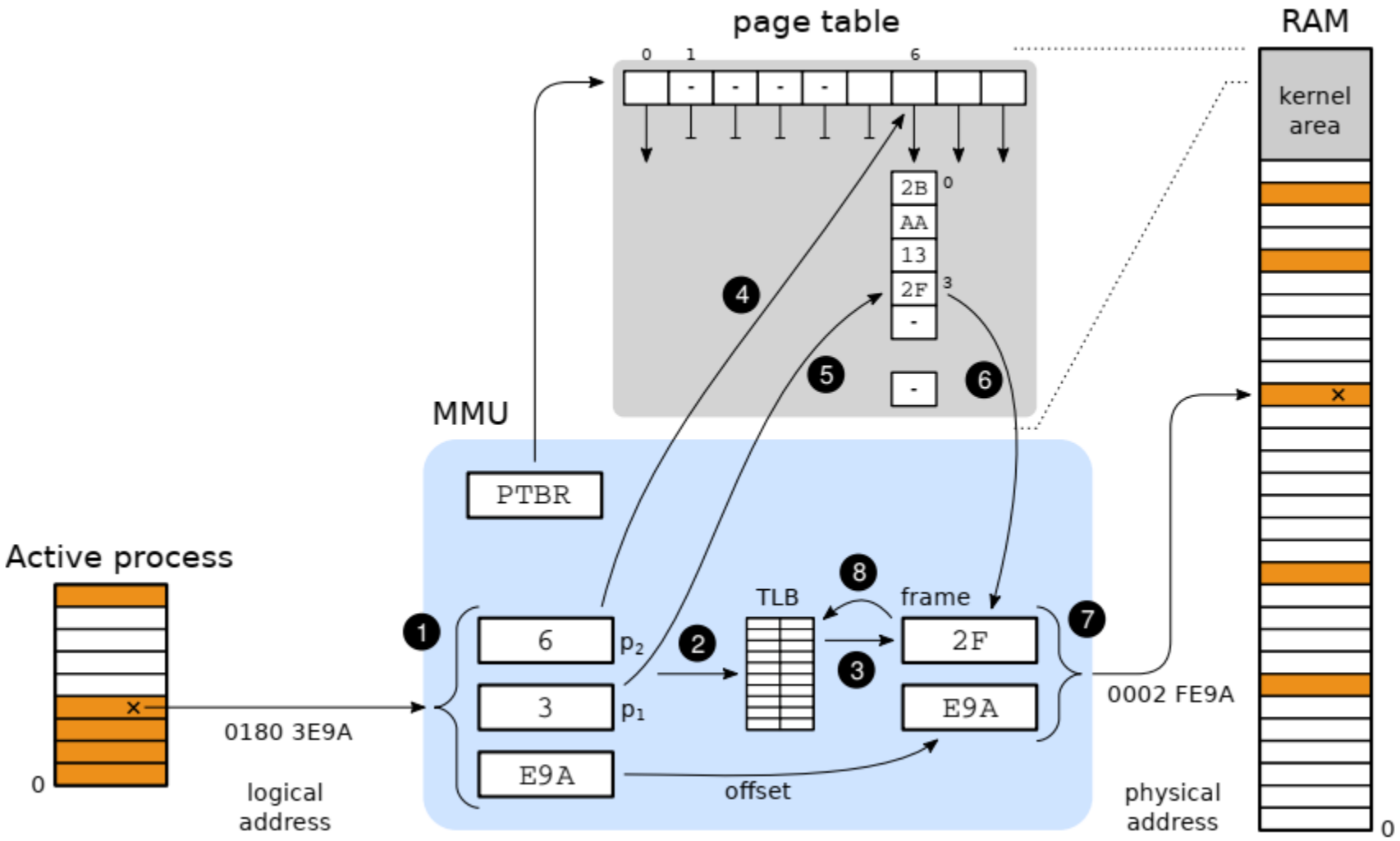


Cache da tabela de páginas - TLB

- A estruturação das tabelas de páginas em vários níveis resolve o problema do espaço ocupado pelas tabelas, mas tem um efeito colateral nocivo: aumenta fortemente o tempo de acesso à memória. Como as tabelas de páginas são armazenadas na memória RAM, cada acesso a um endereço de memória implica em mais acessos para percorrer a árvore de tabelas e encontrar o número de quadro desejado. Em um sistema com tabelas de dois níveis, cada acesso à memória solicitado pelo processador implica em mais dois acessos, para percorrer os dois níveis de tabelas. Com isso, o tempo efetivo de acesso à memória se torna três vezes maior.

- Para atenuar esse problema, consultas recentes à tabela de páginas podem ser armazenadas em um cache dentro da própria MMU, evitando ter de repeti-las e assim diminuindo o tempo de acesso à memória RAM. O cache de tabela de páginas na MMU, denominado TLB (*Translation Lookaside Buffer*) ou *cache associativo*, armazena pares [*página*, *quadro*] obtidos em consultas recentes às tabelas de páginas do processo ativo. Esse cache funciona como uma tabela de *hash*: dado um número de página *p* em sua entrada, ele apresenta em sua saída o número de quadro *q* correspondente, ou um erro, caso não contenha informação sobre *p*.

MMU com paginação e TLB



Taxa de acertos em uma TLB

- Quanto maior a taxa de acertos do TLB (*TLB hit ratio*), melhor é o desempenho dos acessos à memória física. A taxa de acertos de um TLB é influenciada por diversos fatores:
  - **Tamanho do TLB:**
  - **Padrão de acessos à memória**
  - **Trocas de contexto**
  - **Política de substituição de entradas**

Tamanho do TLB:

- quanto mais entradas houverem no TLB, melhor será sua taxa de acerto. Contudo, trata-se de um hardware caro e volumoso, por isso os processadores atuais geralmente têm TLBs com poucas entradas (geralmente entre 16 e 256 entradas). Por exemplo, a arquitetura *Intel 80386* tinha um TLB com 64 entradas para páginas de dados e 32 entradas para páginas de código; por sua vez, o *Intel Core i7* possui 96 entradas para páginas de dados e 142 entradas para páginas de código.

Padrão de acessos à memória:

- processos que concentram seus acessos em poucas páginas de cada vez farão um uso eficiente do TLB, enquanto processos que acessam muitas páginas distintas em um curto período irão gerar frequentes erros de TLB, prejudicando seu desempenho no acesso à memória.

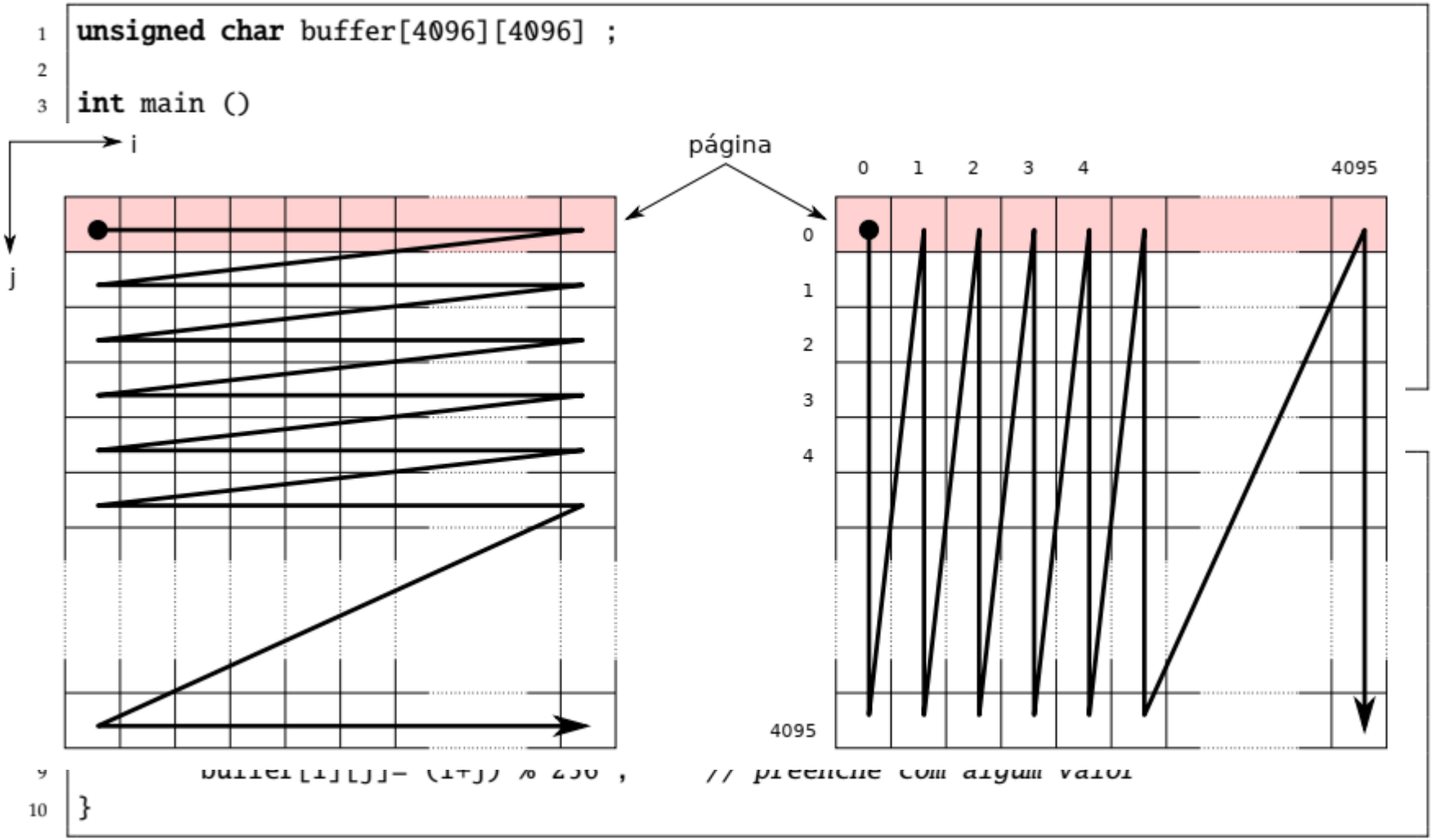
Trocas de contexto:

- o conteúdo do TLB reflete a tabela de páginas do processo ativo em um dado momento. A cada troca de contexto, a tabela de páginas é substituída e portanto o TLB deve ser esvaziado, pois seu conteúdo não é mais válido. Em consequência, trocas de contexto muito frequentes prejudicam a eficiência de acesso à memória, tornando o sistema mais lento.

Política de substituição de entradas:

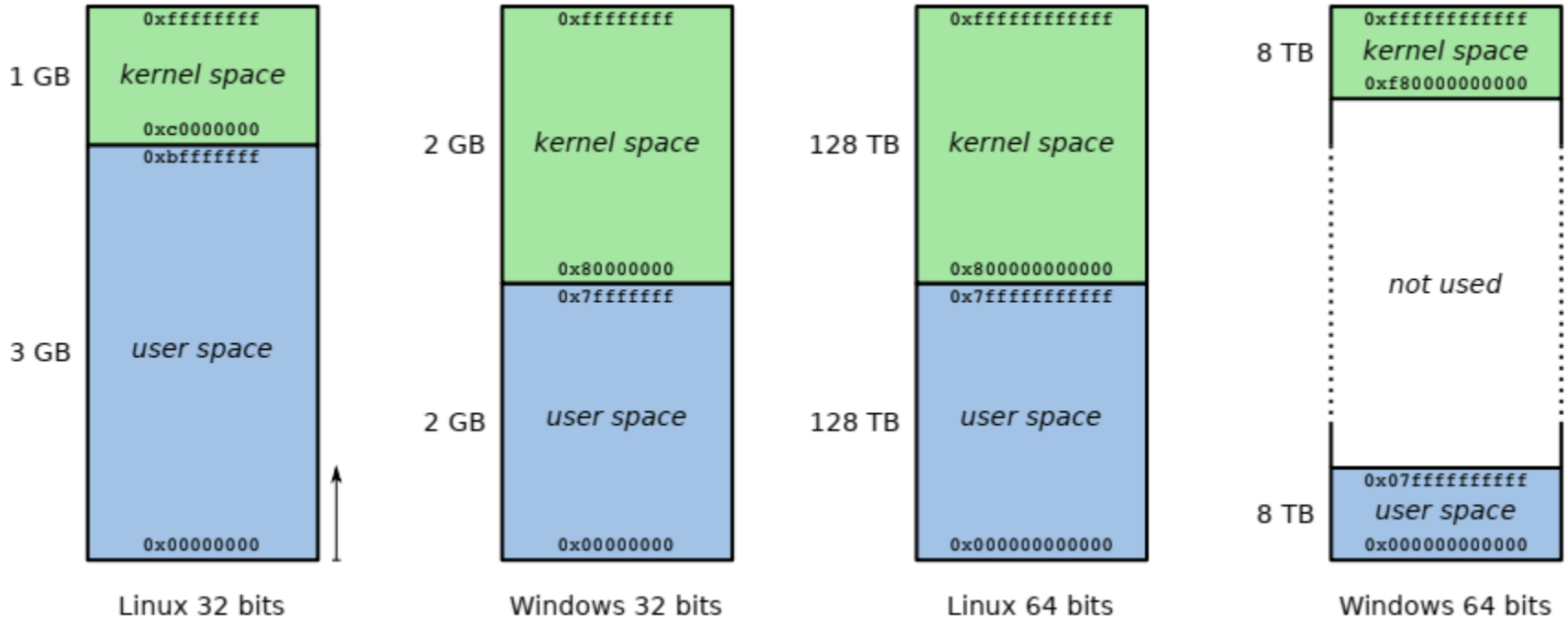
- o que ocorre quando há um erro de TLB e não há mais entradas livres no TLB? Em alguns processadores, a associação [*página*, *quadro*] que gerou o erro é adicionada ao TLB, substituindo a entrada mais antiga; todavia, na maioria dos processadores mais recentes, cada erro de TLB provoca uma interrupção, que transfere ao sistema operacional a tarefa de gerenciar o conteúdo do TLB [Patterson and Henessy, 2005].

Localidade de referências



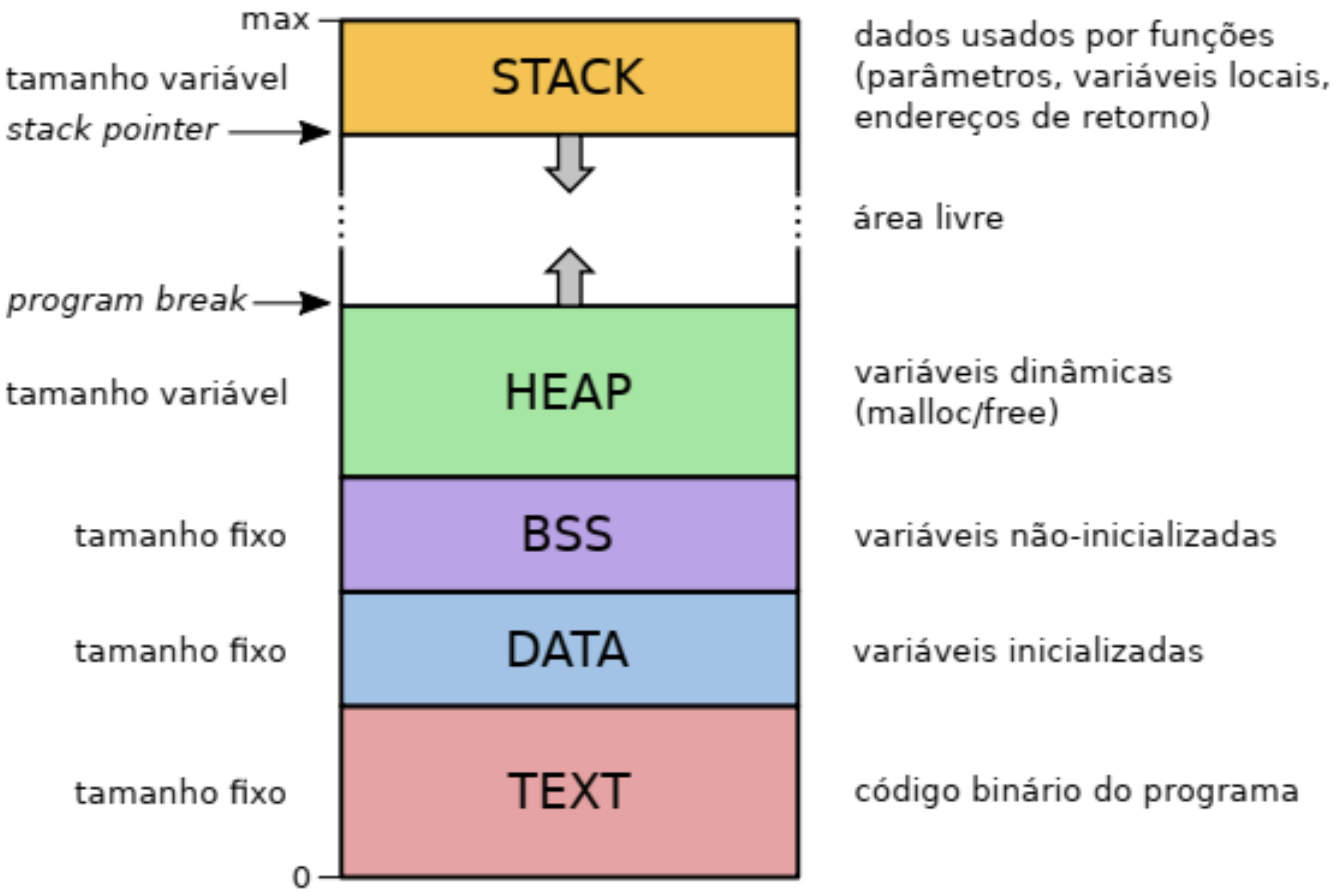
Uso da Memória

- Espaço de endereçamento virtual de um processo



Uso da Memória

Memória de um processo



Uso da Memória

- Comando pmap do Linux

```
1  # pmap 27505
2
3  27505:   /usr/bin/hello
4  Address           Kbytes     Mode    Mapping
5  0000000000400000     808     r-x--   /usr/bin/hello          (TEXT)
6  00000000006c9000      12     rw---   /usr/bin/hello          (DATA)
7  00000000006cc000       8     rw---   [ anon ]                (BSS)
8  000000000092e000     140     rw---   [ anon ]                (HEAP)
9  00007ffe6a5df000     132     rw---   [ stack ]               (STACK)
```

Uso da Memória

- Alocação de variáveis

```
1  #include <stdio.h>
2
3  long int fatorial (int n)
4  {
```

```
1  char * prt ;                  // ponteiro para caracteres
2
3  ptr = malloc (4096) ;         // solicita um bloco de 4.096 bytes;
4                                // ptr aponta para o início do bloco
5
6  if (!ptr)                     // se ptr for nulo, ocorreu um erro
7    abort () ;                  // e a área não foi alocada
8
9  ...                           // usa ptr para acessar o bloco alocado
10
11 free (ptr) ;                  // libera o bloco alocado na linha 3
```

```
17 {
18   printf ("Fatorial (4) = %ld\n", fatorial (4)) ;
19   return 0 ;
20 }
```

Atribuições de endereços

```c
#include <stdlib.h>
#include <stdio.h>

int soma = 0 ;

int main ()
{
   int i ;

   for (i=0; i< 10; i++)
   {
      soma += i ;
      printf ("i vale %d e soma vale %d\n", i, soma) ;
   }
   exit(0) ;
}
```

```
0000000000000000 <main>:
   0:   55                          push   %rbp
   1:   48 89 e5                    mov    %rsp,%rbp
   4:   48 83 ec 10                 sub    $0x10,%rsp
   8:   c7 45 fc 00 00 00 00        movl   $0x0,-0x4(%rbp)
   f:   eb 2f                       jmp    40 <main+0x40>
  11:   8b 15 00 00 00 00           mov    0x0(%rip),%edx
  17:   8b 45 fc                    mov    -0x4(%rbp),%eax
  1a:   01 d0                       add    %edx,%eax
  1c:   89 05 00 00 00 00           mov    %eax,0x0(%rip)
  22:   8b 15 00 00 00 00           mov    0x0(%rip),%edx
  28:   8b 45 fc                    mov    -0x4(%rbp),%eax
  2b:   89 c6                       mov    %eax,%esi
  2d:   bf 00 00 00 00              mov    $0x0,%edi
  32:   b8 00 00 00 00              mov    $0x0,%eax
  37:   e8 00 00 00 00              callq  3c <main+0x3c>
  3c:   83 45 fc 01                 addl   $0x1,-0x4(%rbp)
  40:   83 7d fc 04                 cmpl   $0x4,-0x4(%rbp)
  44:   7e cb                       jle    11 <main+0x11>
  46:   bf 00 00 00 00              mov    $0x0,%edi
  4b:   e8 00 00 00 00              callq  50 <main+0x50>
```

Atribuições de endereços

- A atribuição de endereços aos nomes simbólicos pode ocorrer em diversos momentos da vida do programa:
  - **Na edição**
  - **Na compilação**
  - **Na ligação**
  - **Na carga**
  - **Na execução**

- **Na edição:** o programador escolhe o endereço de cada uma das variáveis e do código do programa na memória. Esta abordagem normalmente só é usada na programação de sistemas embarcados simples, programados diretamente em *Assembly*.

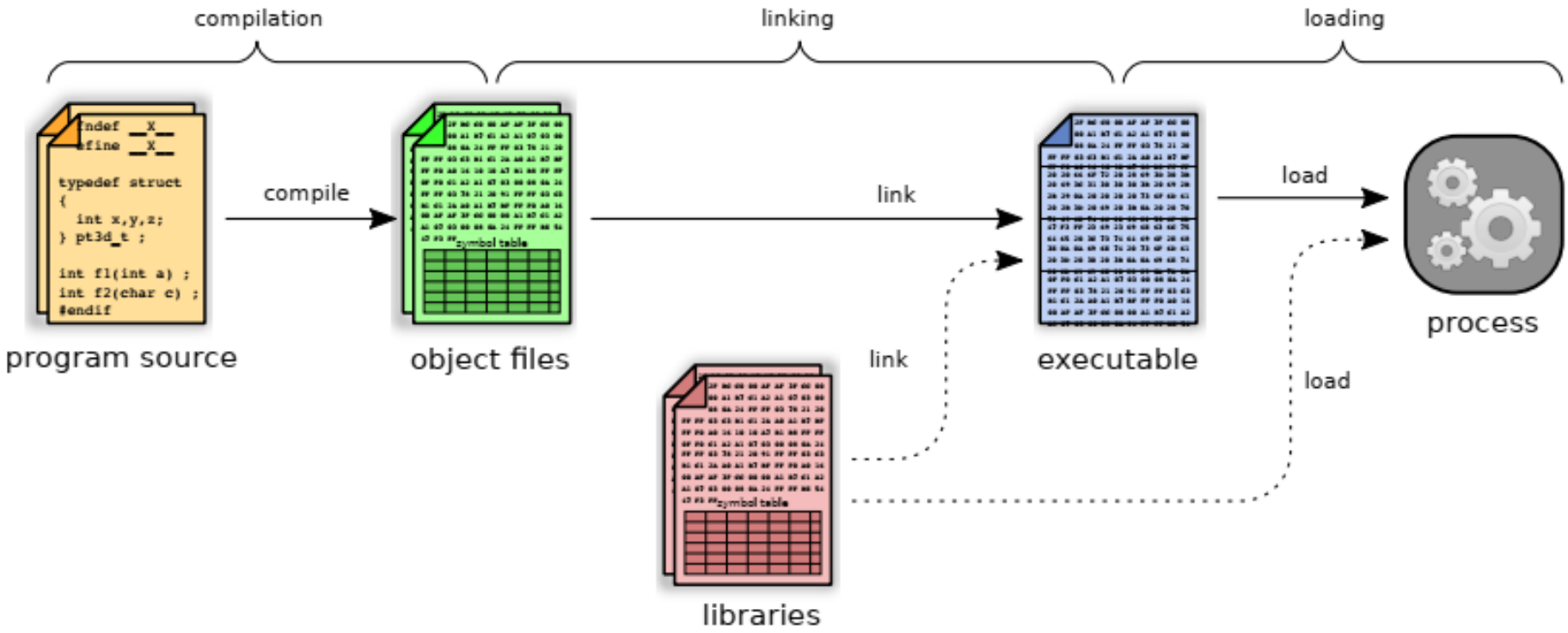
- **Na compilação:** ao traduzir o código-fonte, o compilador escolhe as posições das variáveis na memória. Para isso, todos os códigos-fontes necessários ao programa devem ser conhecidos no momento da compilação, para evitar conflitos de endereços entre variáveis em diferentes arquivos ou bibliotecas. Essa restrição impede o uso de bibliotecas pré-compiladas. Esta abordagem era usada em programas executáveis com extensão .COM do MS-DOS e Windows.

- **Na ligação:** na fase de compilação, o compilador traduz o código fonte em código binário, mas não define os endereços das variáveis e funções, gerando como saída um *arquivo objeto* (*object file*), que contém o código binário e uma *tabela de símbolos* descrevendo as variáveis e funções usadas, seus tipos, onde estão definidas e onde são usadas. A seguir, o ligador (*linker*) pega os arquivos objetos com suas tabelas de símbolos, define os endereços de memória dos símbolos e gera o programa executável [Levine, 2000].

- **Na carga:** também é possível definir os endereços de variáveis e de funções durante a carga do código em memória para o lançamento de um novo processo. Nesse caso, um *carregador* (*loader*) é responsável por carregar o código do processo na memória e definir os endereços de memória que devem ser utilizados. O carregador pode ser parte do núcleo do sistema operacional ou uma biblioteca ligada ao executável, ou ambos. Esse mecanismo normalmente é usado na carga de bibliotecas dinâmicas (DLL - *Dynamic Linking Libraries*).

- **Na execução:** os endereços emitidos pelo processador durante a execução do processo são analisados e convertidos nos endereços efetivos a serem acessados na memória real. Por exigir a análise e a conversão de cada endereço gerado pelo processador, este método só é viável com o auxílio do hardware.

Atribuições de endereços



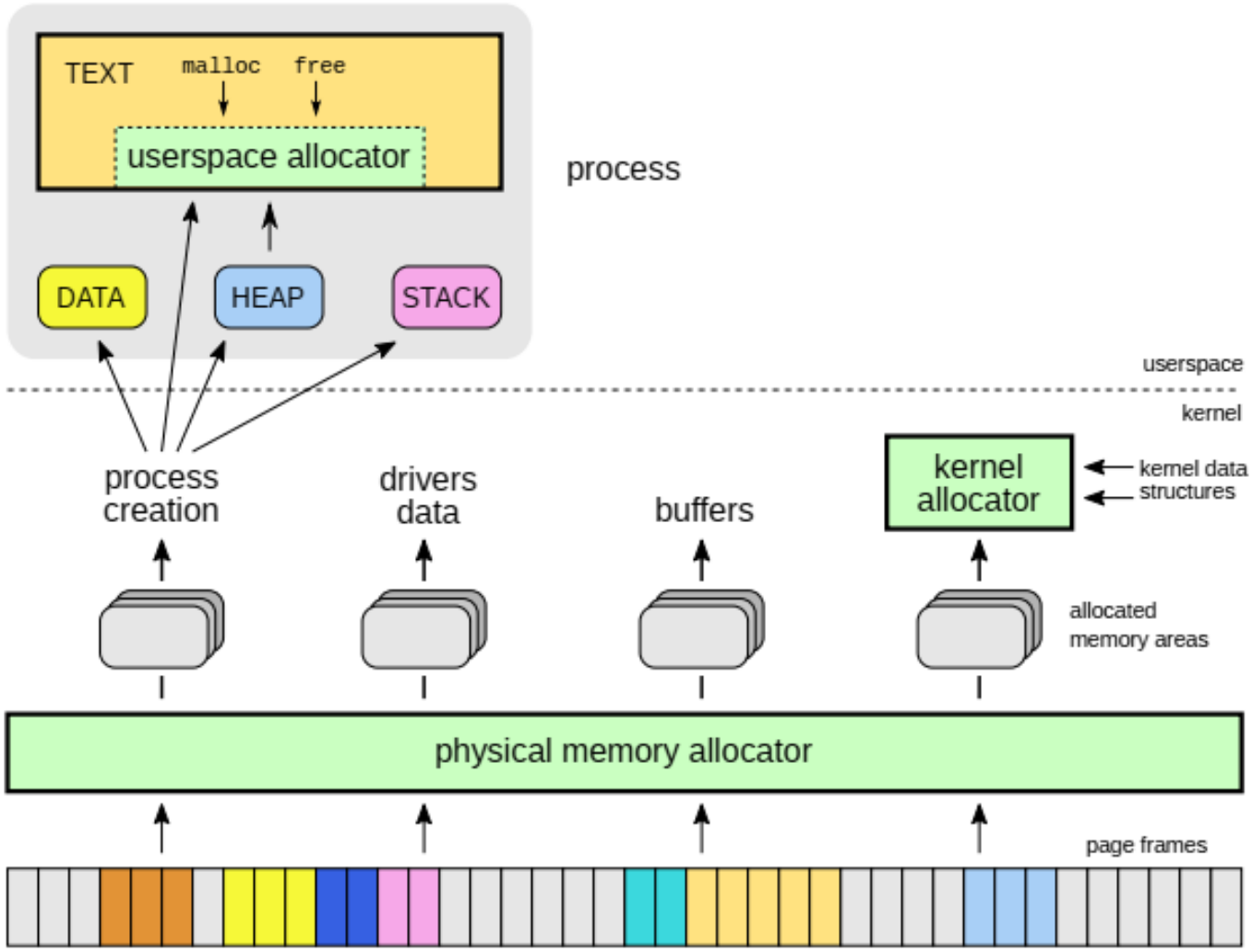
Alocadores de memória

- Como ocorre com os demais recursos de hardware, a memória disponível no sistema deve ser gerenciada pelo SO, para evitar conflitos entre aplicações e garantir justiça no seu uso. Fazendo uso dos mecanismos de hardware de memória apresentados anteriormente, o sistema operacional aloca e libera áreas de memória para os processos (ou para o próprio núcleo), conforme a necessidade.

- o alocador deve manter um registro contínuo de quais áreas estão sendo usadas e quais estão livres. Para ser eficiente, ele deve realizar a alocações rapidamente e minimizar o desperdício de memória.

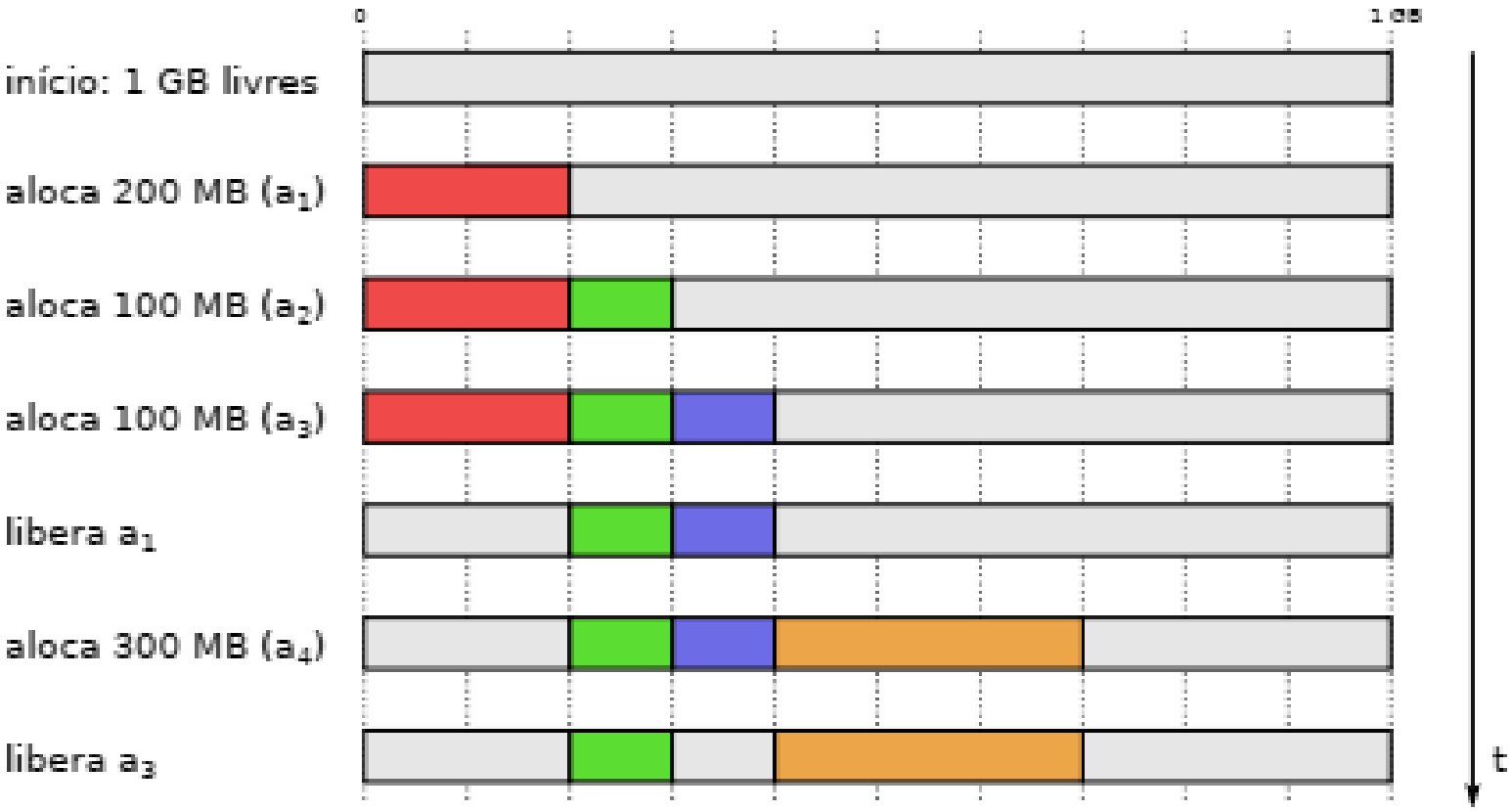
Alocadores de Memória

- Alocadores de memória podem existir em diversos contextos:
  - **Alocador de memória física** : organiza a memória física do computador, definindo as áreas que podem ser alocadas pelo núcleo e processos e as áreas reservadas (BIOS, DMA, etc.). Gerencia o uso das áreas alocáveis, atendendo requisições de alocação/liberação do núcleo e de processos.
  - **Alocador de espaço de núcleo**: o núcleo do SO continuamente cria e destrói muitas estruturas de dados relativamente pequenas, como descritores de arquivos abertos, de processos, *sockets* de rede, *pipes*, etc. O alocador de núcleo obtém áreas de memória do alocador físico e as utiliza para alocar essas estruturas para o núcleo.
  - **Alocador de espaço de usuário**: um processo pode solicitar blocos de memória para armazenar estruturas de dados dinâmicas, através de operações como malloc e free. O alocador de memória do processo geralmente é implementado por bibliotecas providas pelo sistema operacional, como a LibC. Essas bibliotecas interagem com o núcleo para solicitar o redimensionamento da seção HEAP do processo quando necessário.


TEXT
malloc
free
userspace allocator
process
DATA
HEAP
STACK
userspace
kernel
process creation
drivers data
buffers
kernel allocator
kernel data structures
allocated memory areas
physical memory allocator
page frames

Alocação Básica

Sequência de alocações e liberações de memória



Fragmentação

- Ao longo da vida de um sistema, áreas de memória são alocadas e liberadas continuamente. Com isso, podem surgir áreas livres (“buracos” na memória) entre as áreas alocadas. No exemplo anterior, pode-se observar que o sistema ainda tem 600 MB de memória livre após a sequência de operações, mas somente requisições de alocação de até 300 MB pode ser aceitas, pois esse é o tamanho da maior área livre contínua disponível. Esse fenômeno se chama ***fragmentação externa***, pois fragmenta a memória livre, fora das áreas alocadas.

- Pode-se enfrentar o problema da fragmentação externa de duas formas: **minimizando** sua ocorrência, através de estratégias de alocação, **desfragmentando** periodicamente a memória do sistema, ou permitindo a fragmentação interna.

Estratégias de alocação

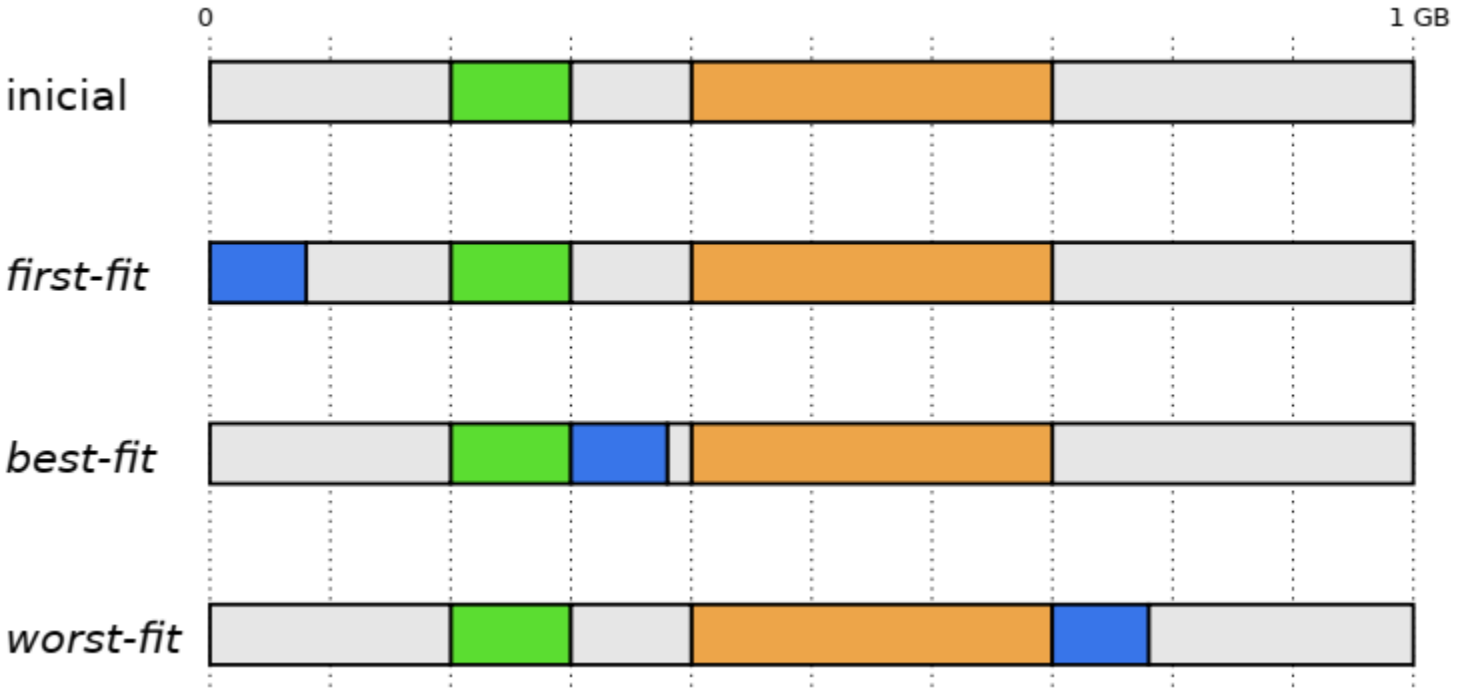
- ***First-fit**** **(primeiro encaixe):** consiste em escolher a primeira área livre que satisfaça o pedido de alocação; tem como vantagem a rapidez, sobretudo se a lista de áreas livres for muito longa.

- ***Best-fit**** **(melhor encaixe):** consiste em escolher a menor área possível que possa receber a alocação, minimizando o desperdício de memória. Contudo, algumas áreas livres podem ficar pequenas demais e com isso se tornarem inúteis.

- ***Worst-fit*** **(pior encaixe):** consiste em escolher sempre a maior área livre possível, de forma que a "sobra" seja grande o suficiente para ser usada em outras alocações.

- ***Next-fit*** **(próximo encaixe):** variante da estratégia *first-fit* que consiste em percorrer a lista de áreas a partir da última área alocada ou liberada, para que o uso das áreas livres seja distribuído de forma mais homogênea no espaço de memória.

Estratégias para minimizar a fragmentação externa

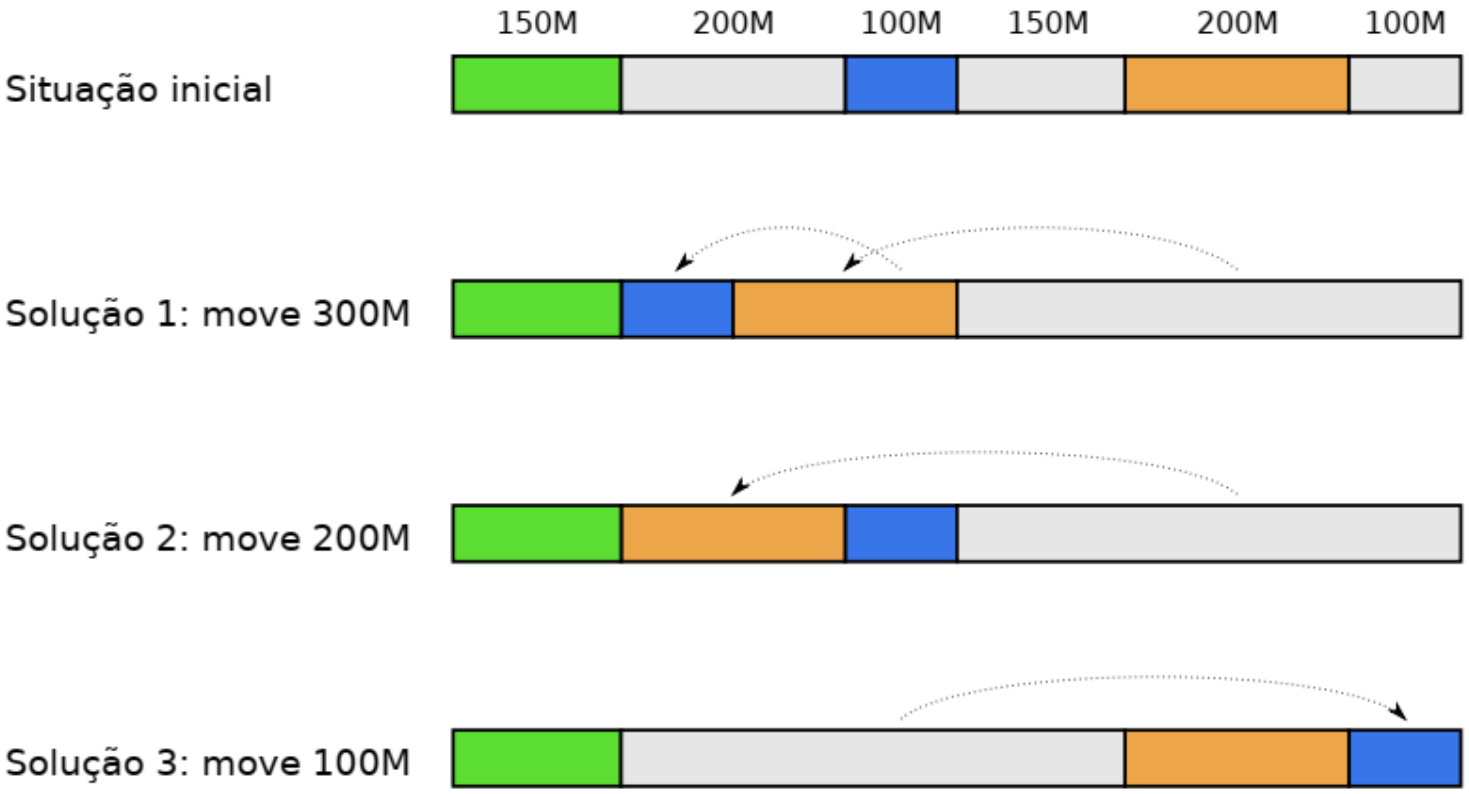


- * Diversas pesquisas demonstraram que as abordagens mais eficientes são a *best-fit* e a *first-fit*, sendo esta última bem mais rápida.

Desfragmentação

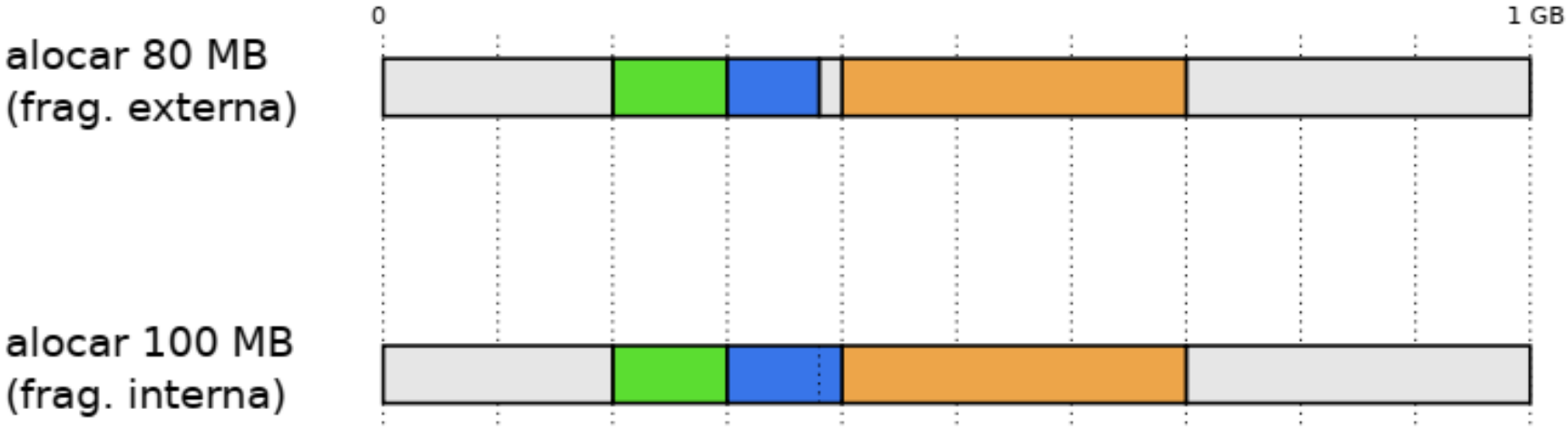
❑ Possibilidades de desfragmentação

(Obs.: As possibilidades de movimentação de áreas podem ser muitas, portanto a desfragmentação deve ser tratada como um problema de otimização combinatória)



Fragmentação interna

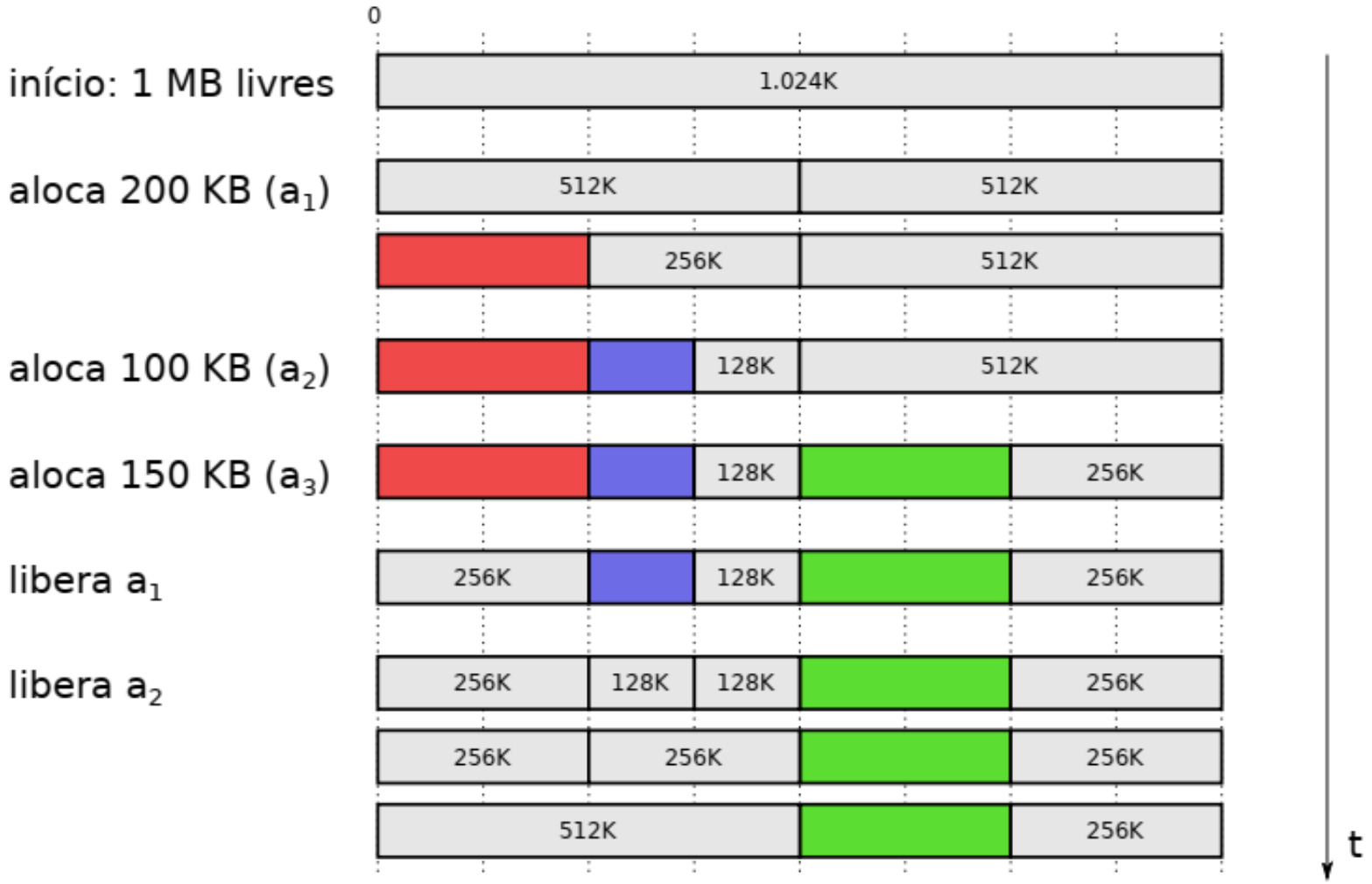
- Uma alternativa para minimizar o impacto da fragmentação externa consiste em arredondar algumas requisições de alocação, para evitar sobras muito pequenas. Por exemplo, na alocação com *best-fit* da Figura 16.3, a área alocada poderia ser arredondada de 80 MB para 100 MB, evitando a sobra da área de 20 MB.



O alocador Buddy

- Existem estratégias de alocação mais sofisticadas e eficientes que as apresentadas anteriormente. Um algoritmo de alocação muito conhecido é o chamado *Buddy Allocator*, ou alocador por pares. Em sua versão mais simples, a estratégia *Buddy* sempre aloca blocos de memória de tamanho 2*n*, com *n* inteiro e ajustável. Por exemplo, para uma requisição de 85 KBytes será alocado um bloco de memória com 128 KBytes (27 KBytes ou 217 bytes), e assim por diante. O uso de blocos de tamanho 2*n* reduz a fragmentação externa, mas pode gerar muita fragmentação interna.

Ex.: Memória de 1 MBytes (1.024 KBytes) e o menor bloco alocável de 64 KBytes.



Paginação em disco

- Os mecanismos de memória virtual suportados pelo hardware, apresentados anteriormente, permitem usar dispositivos de armazenamento secundário como extensão da memória RAM. Com isso, partes ociosas da memória podem ser transferidas para um disco, liberando a memória RAM para outros usos. Caso algum processo tente acessar esse conteúdo posteriormente, ele deverá ser trazido de volta à memória. A transferência dos dados entre memória e disco é feita pelo sistema operacional, de forma transparente para os processos.

Estendendo a Memória RAM

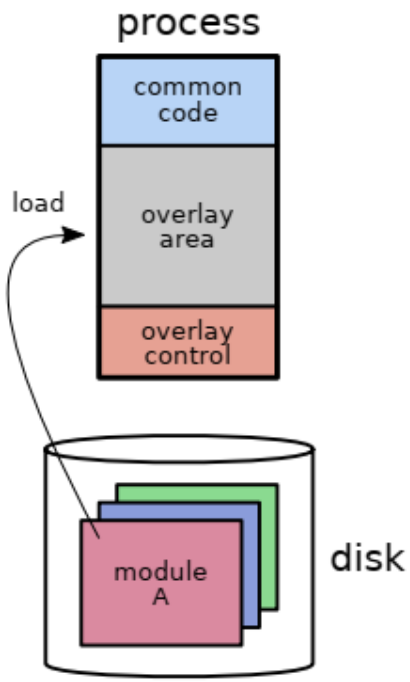
- Existem diversas técnicas para usar um espaço de armazenamento secundário como extensão da memória RAM, com ou sem o auxílio do hardware. As mais conhecidas são:

- **Overlays:** o programador organiza seu programa em módulos que serão carregados em uma mesma região de memória em momentos distintos. Esses módulos, chamados de *overlays*, são gerenciados através de uma biblioteca específica. Por exemplo, o código de um compilador pode separar o analisador léxico, o analisador sintático e o gerador de código em overlays, que serão ativados em momentos distintos.

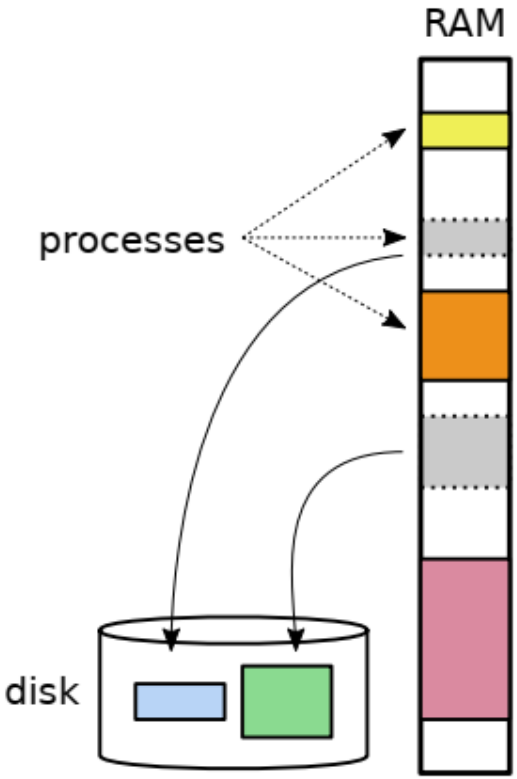
- **Swapping:** consiste em mover um processo ocioso da memória RAM para um disco (*swap-out*), liberando a memória para outros processos. Mais tarde, quando esse processo for acordado (entrar na fila de prontos do escalonador), ele é carregado de volta na memória (*swap-in*). A técnica de *swapping* foi muito usada até os anos 1990, mas hoje é pouco empregada em sistemas operacionais de uso geral.

- **Paging:** consiste em mover páginas individuais, conjuntos de páginas ou mesmo segmentos da memória para o disco (*page-out*). Se o processo tentar acessar uma dessas páginas mais tarde, a MMU gera uma interrupção de falta de página e o núcleo do SO recarrega a página faltante na memória (*page-in*). Esta é a técnica mais usada nos sistemas operacionais atuais, por sua flexibilidade, rapidez e eficiência.

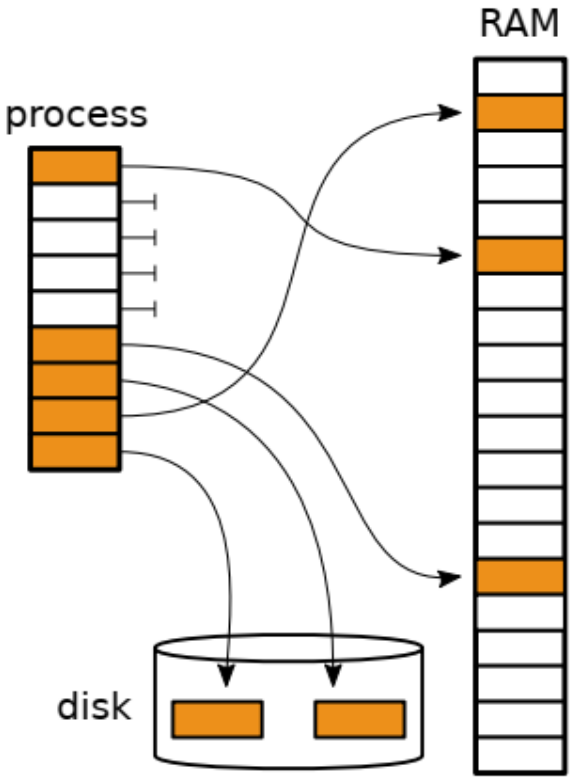
Estendendo a Memória RAM



overlay

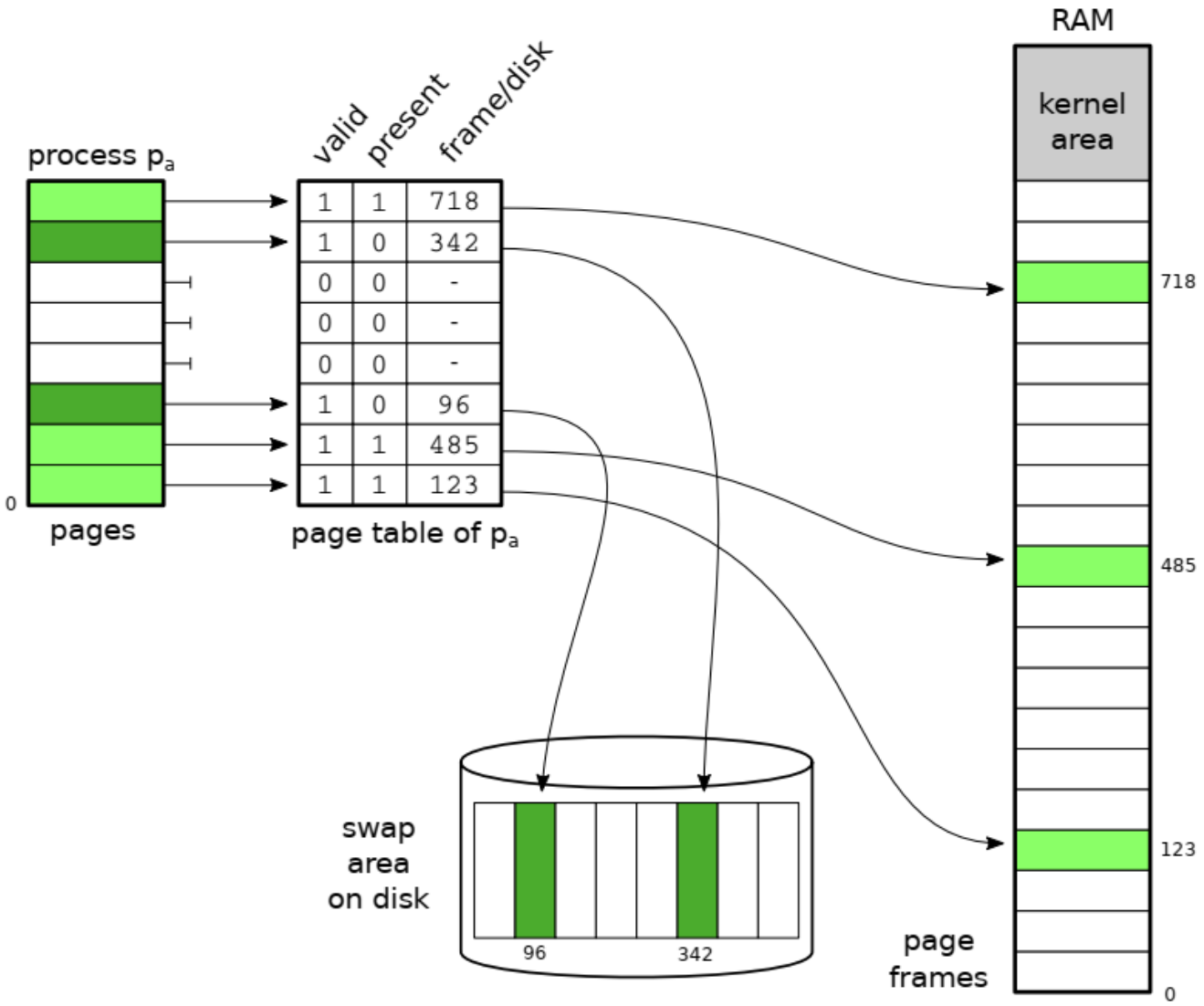


swapping

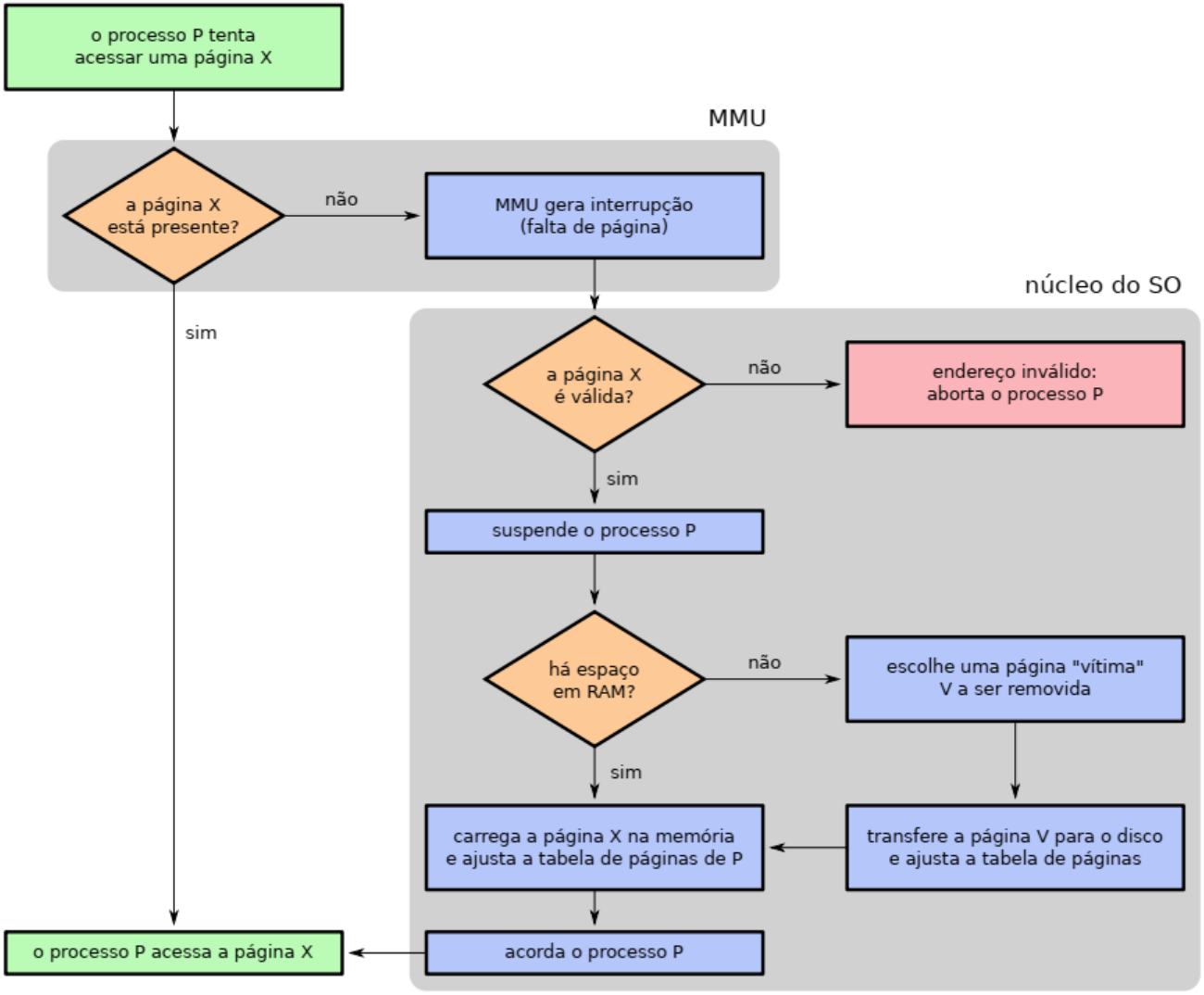


paging

Paginação em disco



Paginação em disco



Critérios de seleção das páginas “vítimas”

- Idade da página
- Frequência de acessos à página
- Data do último acesso
- Prioridade do processo proprietário
- Conteúdo da página
- Páginas especiais

Algoritmos clássicos

- **Cadeias de referências:** indica a sequência de páginas acessadas por um processo ao longo de sua execução, considerando todos os endereços acessados pelo processo nas várias áreas de memória que o compõem (código, dados, pilha, *heap*, etc).
- Para possibilitar a comparação dos algoritmos de substituição de páginas apresentados na sequência, será usada a seguinte cadeia de referências fictícia, com 20 acessos a 6 páginas distintas, obtida de [Silberschatz et al., 2001]:

  7; 0; 1; 2; 0; 3; 0; 4; 2; 3; 0; 3; 2; 1; 2; 0; 1; 7; 0; 1

  e memória física com 3 quadros

Algoritmo FIFO

t	página acessada	quadros q_o	q_1	q_2	falta de página?	ação realizada
0						situação inicial, quadros vazios
1	7	7			✓	p_7 é carregada em q_0
2	0	7	0		✓	p_0 é carregada em q_1
3	1	7	0	1	✓	p_1 é carregada em q_2
4	2	2	0	1	✓	p_2 substitui p_7 (carregada em $t = 1$)
5	0	2	0	1		p_0 já está na memória
6	3	2	3	1	✓	p_3 substitui p_0
7	0	2	3	0	✓	p_0 substitui p_1
8	4	4	3	0	✓	p_4 substitui p_2
9	2	4	2	0	✓	p_2 substitui p_3
10	3	4	2	3	✓	p_3 substitui p_0
11	0	0	2	3	✓	p_0 substitui p_4
12	3	0	2	3		p_3 já está na memória
13	2	0	2	3		p_2 já está na memória
14	1	0	1	3	✓	p_1 substitui p_2
15	2	0	1	2	✓	p_2 substitui p_3
16	0	0	1	2		p_0 já está na memória
17	1	0	1	2		p_1 já está na memória
18	7	7	1	2	✓	p_7 substitui p_0
19	0	7	0	2	✓	p_0 substitui p_1
20	1	7	0	1	✓	p_1 substitui p_2

Algoritmo Ótimo

t	página acessada	quadros q_o	q_1	q_2	falta de página?	ação realizada
0						situação inicial, quadros vazios
1	7	7			✓	p_7 é carregada em q_0
2	0	7	0		✓	p_0 é carregada em q_1
3	1	7	0	1	✓	p_1 é carregada em q_2
4	2	2	0	1	✓	p_2 substitui p_7 (que só será acessada em $t = 18$)
5	0	2	0	1		p_0 já está na memória
6	3	2	0	3	✓	p_3 substitui p_1
7	0	2	0	3		p_0 já está na memória
8	4	2	4	3	✓	p_4 substitui p_0
9	2	2	4	3		p_2 já está na memória
10	3	2	4	3		p_3 já está na memória
11	0	2	0	3	✓	p_0 substitui p_4
12	3	2	0	3		p_3 já está na memória
13	2	2	0	3		p_2 já está na memória
14	1	2	0	1	✓	p_1 substitui p_3
15	2	2	0	1		p_2 já está na memória
16	0	2	0	1		p_0 já está na memória
17	1	2	0	1		p_1 já está na memória
18	7	7	0	1	✓	p_7 substitui p_2
19	0	7	0	1		p_0 já está na memória
20	1	7	0	1		p_1 já está na memória

Algoritmo LRU

t	página acessada	quadros q_o	q_1	q_2	falta de página?	ação realizada
0						situação inicial, quadros vazios
1	7	7			✓	p_7 é carregada em q_0
2	0	7	0		✓	p_0 é carregada em q_1
3	1	7	0	1	✓	p_1 é carregada em q_2
4	2	2	0	1	✓	p_2 substitui p_7 (há mais tempo sem acesso)
5	0	2	0	1		p_0 já está na memória
6	3	2	0	3	✓	p_3 substitui p_1
7	0	2	0	3		p_0 já está na memória
8	4	4	0	3	✓	p_4 substitui p_2
9	2	4	0	2	✓	p_2 substitui p_3
10	3	4	3	2	✓	p_3 substitui p_0
11	0	0	3	2	✓	p_0 substitui p_4
12	3	0	3	2		p_3 já está na memória
13	2	0	3	2		p_2 já está na memória
14	1	1	3	2	✓	p_1 substitui p_0
15	2	1	3	2		p_2 já está na memória
16	0	1	0	2	✓	p_0 substitui p_3
17	1	1	0	2		p_1 já está na memória
18	7	1	0	7	✓	p_7 substitui p_2
19	0	1	0	7		p_0 já está na memória
20	1	1	0	7		p_1 já está na memória

Conjunto de trabalho (*Working Set ou WS*)

- A localidade de referências (estudada na Seção 14.9) mostra que os processos normalmente acessam apenas uma pequena fração de suas páginas a cada instante. O conjunto de páginas acessadas na história recente de um processo é chamado *Conjunto de Trabalho* (*Working Set*, ou *ws*). A composição do conjunto de trabalho é dinâmica, variando à medida em que o processo executa e evolui seu comportamento, acessando novas páginas e deixando de acessar outras.

Exemplos de conjuntos de trabalho

t	página	$ws(n = 3)$	$ws(n = 4)$	$ws(n = 5)$
1	7	7	7	7
2	0	0, 7	0, 7	0, 7
3	1	0, 1, 7	0, 1, 7	0, 1, 7
4	2	0, 1, 2	0, 1, 2, 7	0, 1, 2, 7
5	0	0, 1, 2	0, 1, 2	0, 1, 2, 7
6	3	0, 2, 3	0, 1, 2, 3	0, 1, 2, 3
7	0	0, 3	0, 2, 3	0, 1, 2, 3
8	4	0, 3, 4	0, 3, 4	0, 2, 3, 4
9	2	0, 2, 4	0, 2, 3, 4	0, 2, 3, 4
10	3	2, 3, 4	0, 2, 3, 4	0, 2, 3, 4
11	0	0, 2, 3	0, 2, 3, 4	0, 2, 3, 4
12	3	0, 3	0, 2, 3	0, 2, 3, 4
13	2	0, 2, 3	0, 2, 3	0, 2, 3
14	1	1, 2, 3	0, 1, 2, 3	0, 1, 2, 3
15	2	1, 2	1, 2, 3	0, 1, 2, 3
16	0	0, 1, 2	0, 1, 2	0, 1, 2, 3
17	1	0, 1, 2	0, 1, 2	0, 1, 2
18	7	0, 1, 7	0, 1, 2, 7	0, 1, 2, 7
19	0	0, 1, 7	0, 1, 7	0, 1, 2, 7
20	1	0, 1, 7	0, 1, 7	0, 1, 7